Principles of Network Applications



Continued…

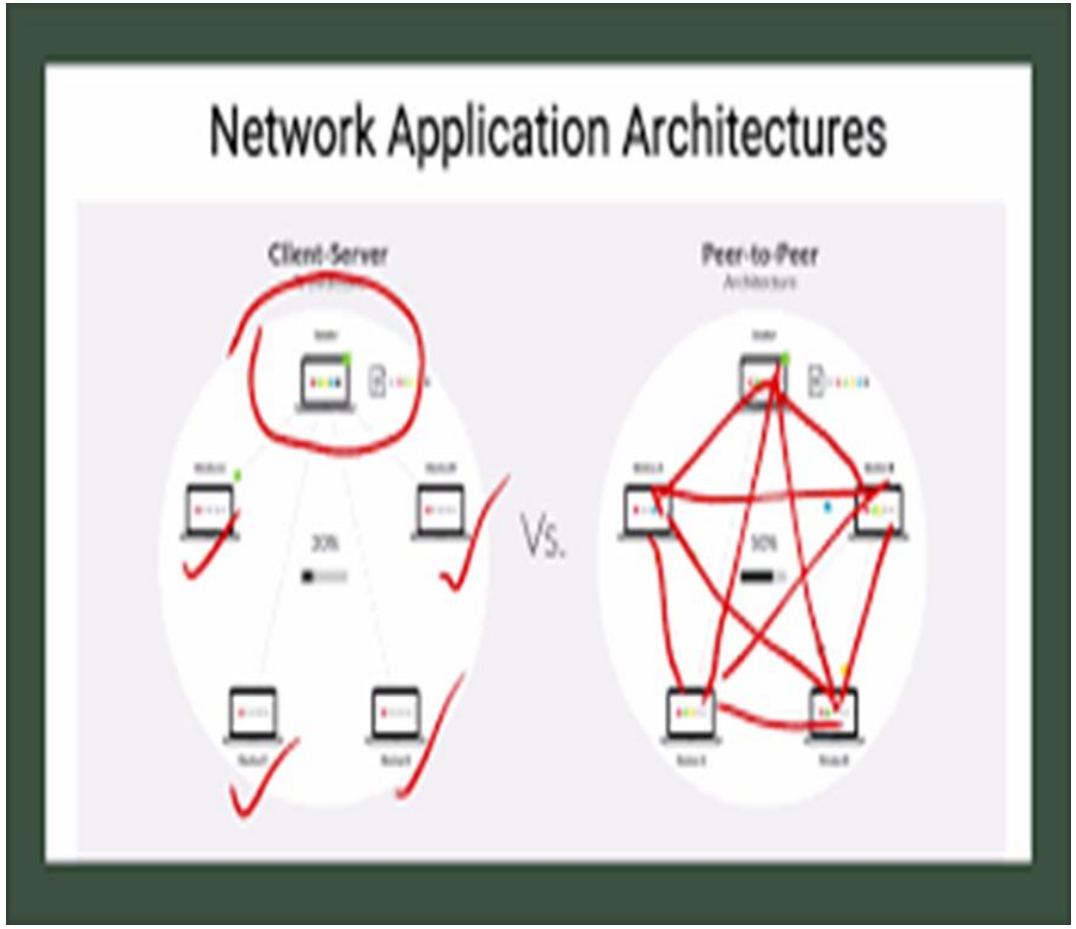
The Principles of Network Applications are fundamental concepts that govern the design and development of applications that run on a computer network. These principles encompass several key aspects of network applications, including:

- Network Application Architectures
- Processes Communicating
- The Interface Between the Process and the Computer Network
- Transport Services Available to Applications
- Transport Services Provided by the Internet
- Application-Layer Protocols

Network Application Architectures

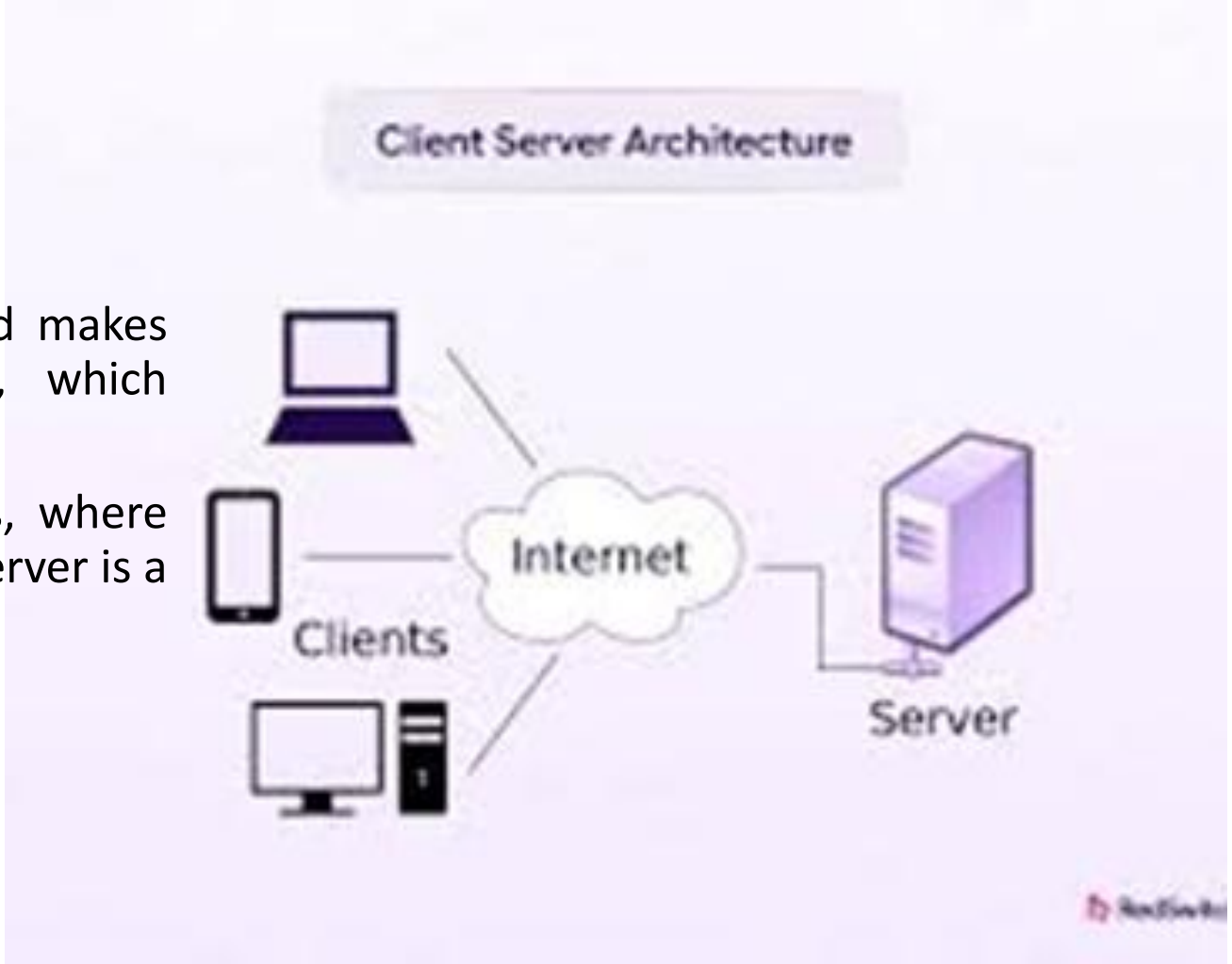
Network Application Architectures refer to the overall design and structure of a network application. It encompasses how the application is divided into different components, and how these components interact with each other. There are several commonly used network application architectures, including:

- **Client-Server Architecture**
- **Peer-to-Peer Architecture**
- **Three-Tier Architecture**
- **Microservices Architecture**



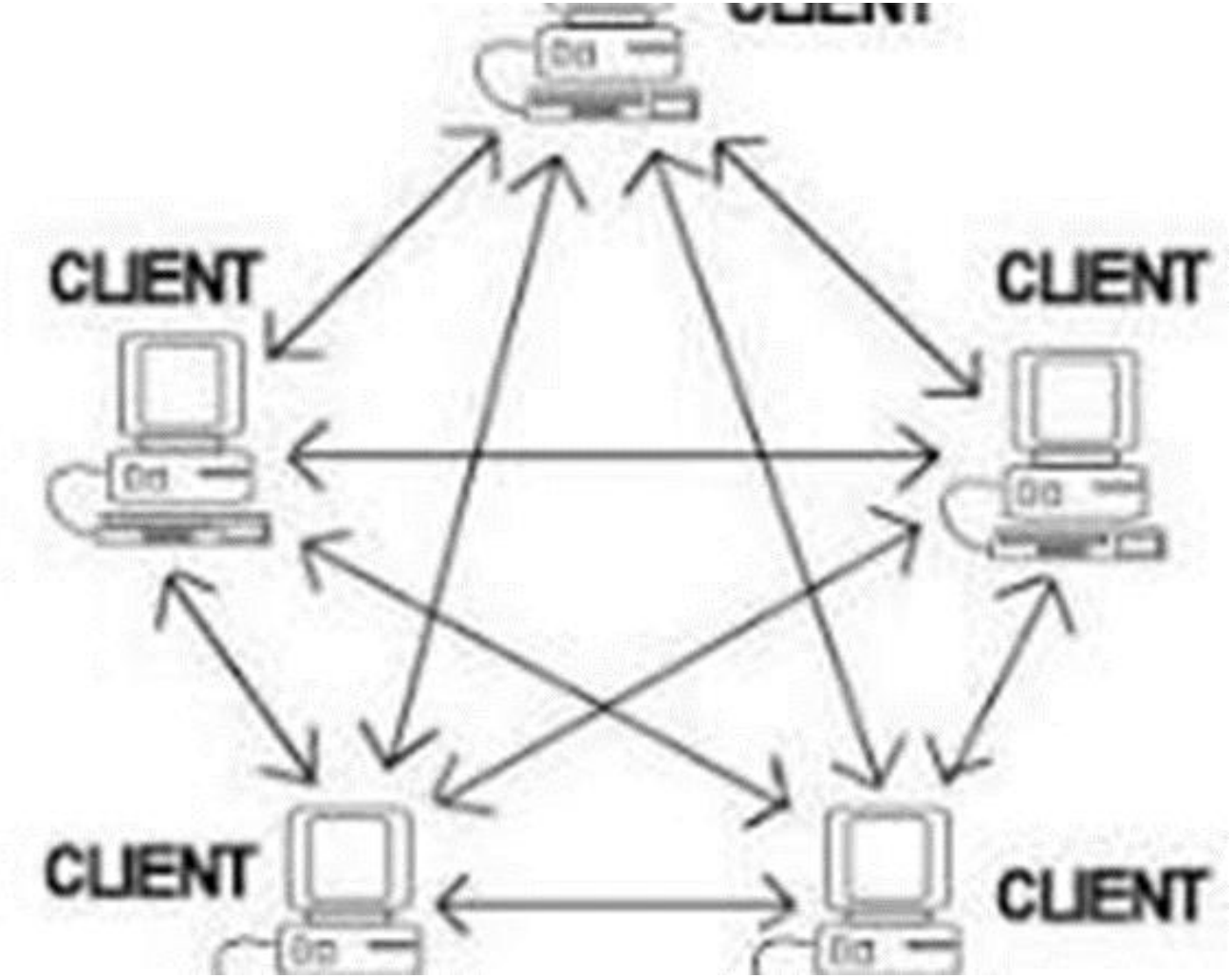
Client-Server Architecture

- one component acts as a client and makes requests to a server component, which provides the requested services.
- commonly used in web applications, where the client is a web browser and the server is a web server.



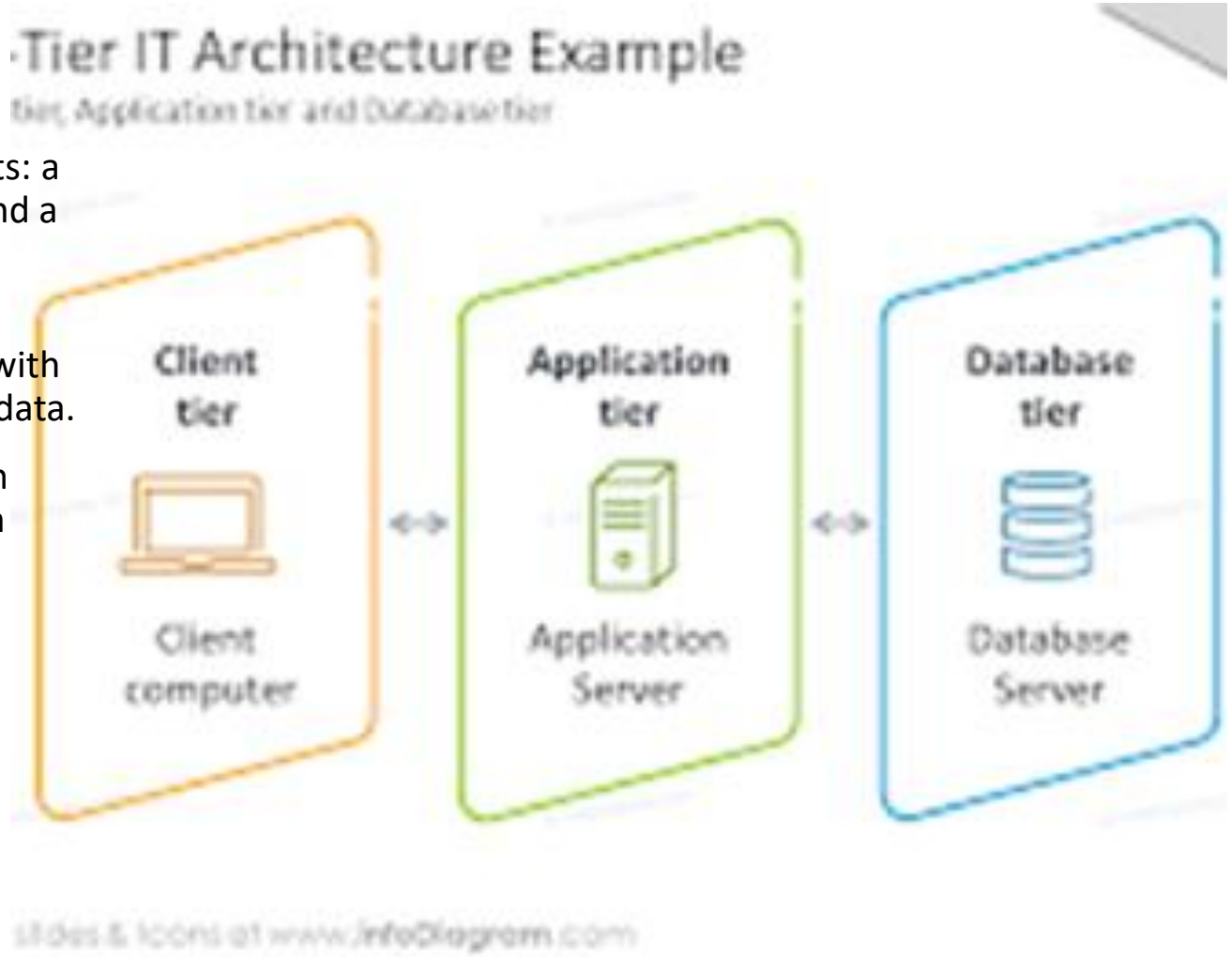
Peer to Peer architecture

- Every component is both a client and a server, and each component can communicate directly with any other component.
- Commonly used in file-sharing applications, where each user's device acts as both a client and a server.



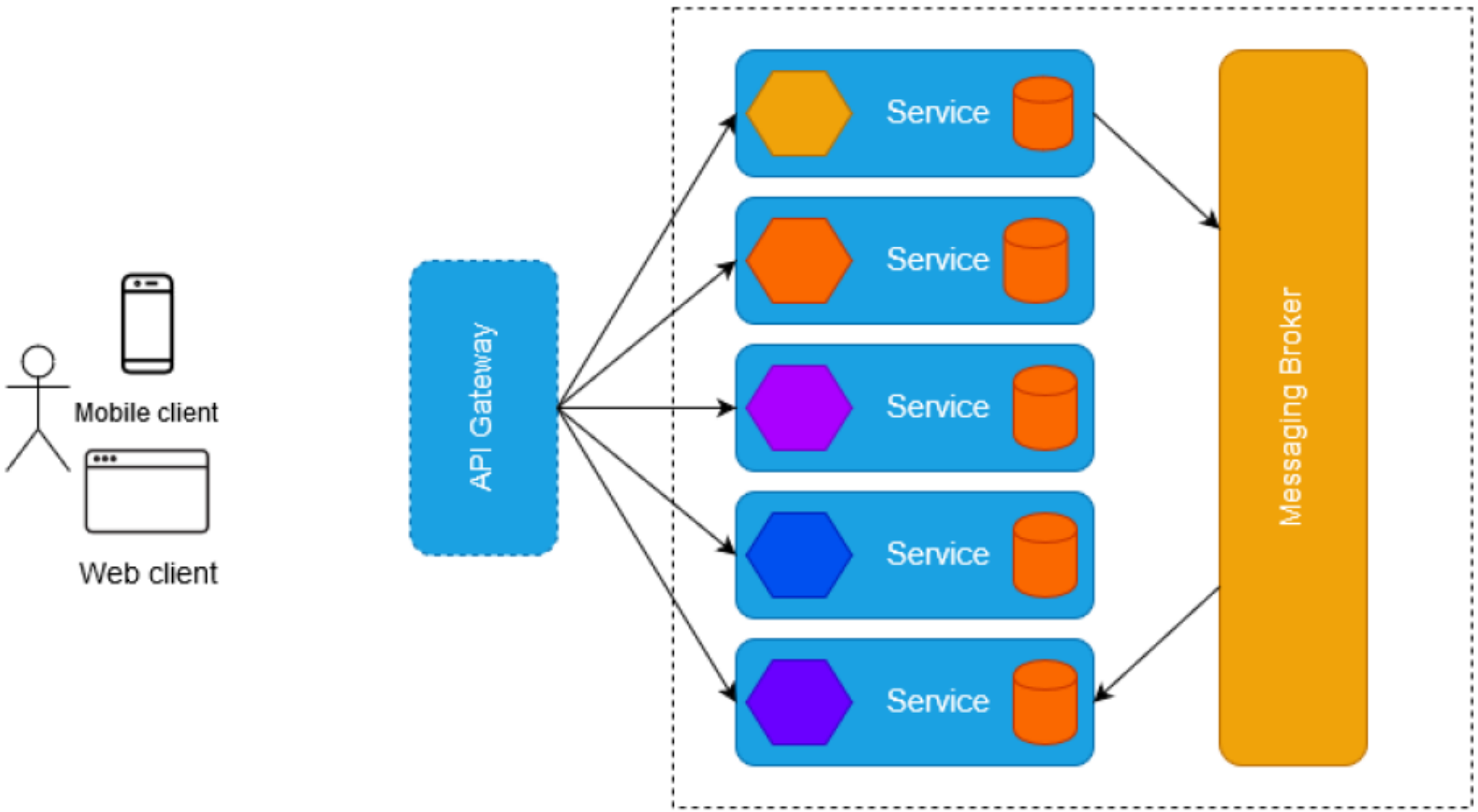
Three-Tier Architecture

- The application is divided into three components: a client component, a middleware component, and a database component.
- The client component makes requests to the middleware component, which communicates with the database component to retrieve or update data.
- This architecture provides a layer of abstraction between the client and the database, which can improve performance and scalability.



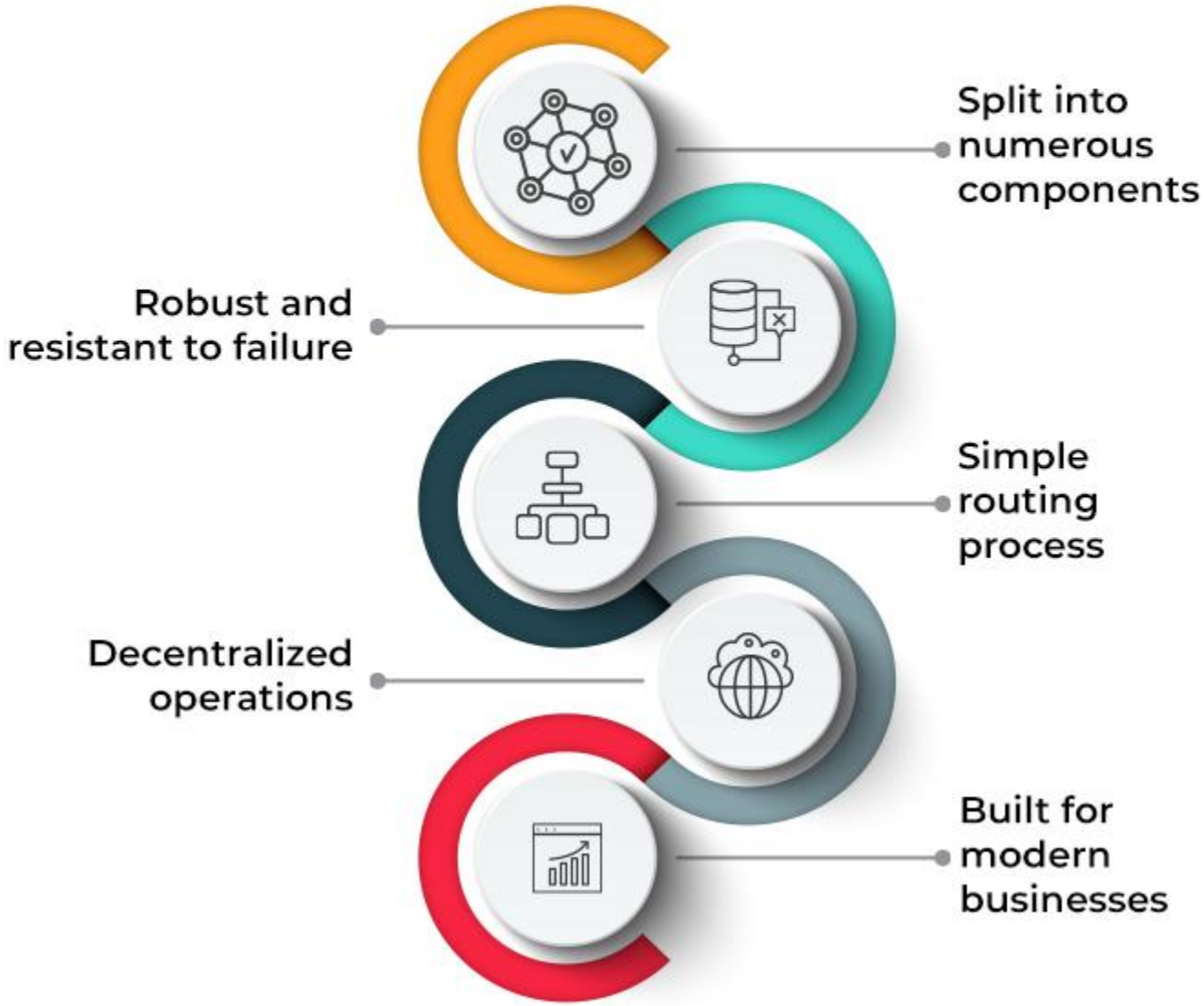
Microservices Architecture

- A microservices architecture is a design approach where an application is structured as a collection of small, independent, and loosely coupled services.
- These services, often called microservices, are independently deployable and can communicate with each other through well-defined APIs.
- Each service performs **one specific task**.
- Each service can be **developed, tested, deployed, and maintained independently**.
- Services communicate with one another using **APIs (Application Programming Interfaces**
- This contrasts with a monolithic architecture where all application components are tightly integrated.
- Leading online companies such as Amazon, eBay, Netflix, PayPal, Twitter, and Uber have dropped monolithic architectures and moved to microservices**.**


Mobile client
Web client
API Gateway
Service
Service
Service
Service
Service
Messaging Broker

TOOLBOX™

CHARACTERISTICS OF MICROSERVICE ARCHITECTURE



1. Split into numerous components

- Software built using a microservices architecture is, by definition, broken down into numerous component services.
- Each service can be created, deployed, and updated independently without compromising application integrity.

2. Robust and resistant to failure

- It is not easy for an application built using a microservices architecture to fail.
- Of course, individual services can fail, undoubtedly affecting operations.
- After all, numerous diverse and unique services communicate with each other to carry out operations in a microservices environment, and failure is bound to occur at some point.
- However, in a correctly configured microservices-based application, a function facing downtime should be able to reroute traffic away from itself while allowing its connected services to continue operating.

3. Simple routing process

- Microservices consist of intelligent components capable of processing data and applying logic.
- These components are connected by ‘dumb wires’ that transmit information from one element to another.
- This simple routing process is the opposite of the architecture used by some other enterprise applications.

4. Decentralized operations

- Microservices leverage numerous platforms and technologies. This makes traditional centralized governance methods inefficient for operating a microservices architecture.
- Decentralized governance is better suited for microservices as developers worldwide create valuable tools to solve operational challenges.
- Similarly, a microservices architecture favors decentralized data management, as every microservice application manages its unique database.

5. Built for modern businesses

- Microservices architecture is created to focus on fulfilling the requirements of modern, digital businesses.
- Microservices rely on cross-functional teams. Each team takes responsibility for creating specific products based on individual services transmitting and receiving data through a message bus.

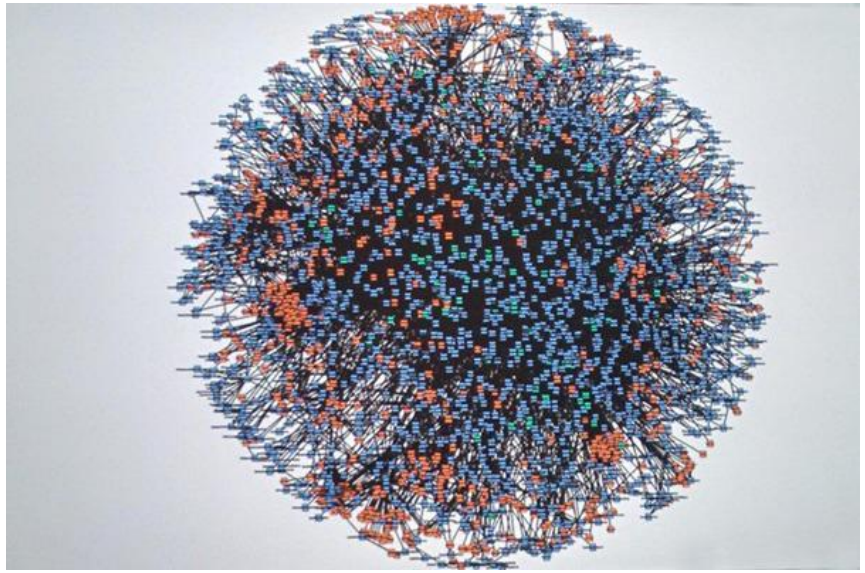
Examples of Microservices

Leading tech companies use microservices for numerous reasons, such as to reduce the complexity of their architecture, speed up development, and make applications more responsive and easier to update.

Listed below are three real-life examples of market leaders adopting microservices to enhance their operations.

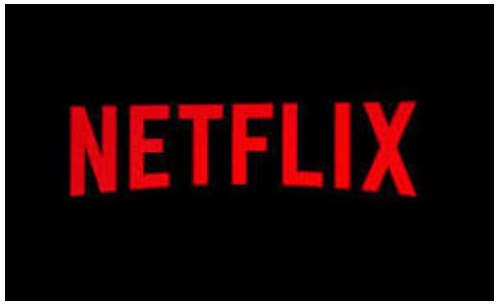
1. Amazon

- ✓ At the turn of the millennium, Amazon's retail website was a monolith with tightly knit connections between and within its multi-tiered services.
- ✓ Amazon divided its monolithic applications into small, independent, specialized applications



2. Netflix

- ✓ Just like Amazon, Netflix is a pioneer in the microservices space.
- ✓ Its migration began in 2008 when the streaming platform faced numerous scaling issues and service outages.
- ✓ When a primary database of the company was corrupted, causing a three-day blockage on DVD shipping to members, Netflix realized it was time to migrate to microservices.
- ✓ In 2009, Netflix started gradually refactoring its monolithic architecture into microservices, one service at a time.
- ✓ It began by migrating its movie-coding platform–which was not user-facing – to operate on the AWS cloud through a standalone microservice architecture.
- ✓ Soon after, it began converting its user-facing systems to microservices, completing the process in 2012.



3. Uber

- ✓ Like Amazon and Netflix, Uber, too, decided to shift away from its monolithic structure due to growth hurdles.
- ✓ The challenges faced by the ride-sharing platform included inefficiency in the development and launch of new features, the inability to fix bugs quickly, and problems with integrating its fast growing global operations.
- ✓ To conquer the challenges presented by its monolithic application, Uber broke it down into cloud-powered microservices.
- ✓ Soon, individual microservices were developed for business functions such as trip management and passenger management.

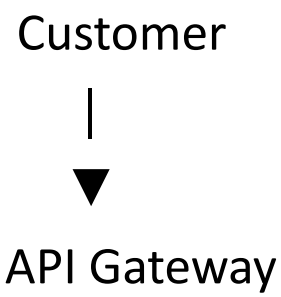


Complete Working Example (Amazon)

Suppose a customer purchases a mobile phone.

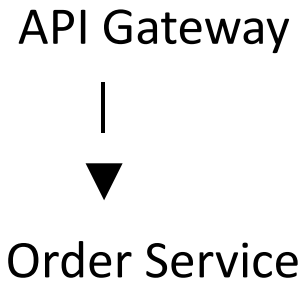
Step 1

Customer clicks **Buy Now**.



Step 2

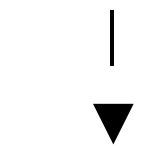
The API Gateway forwards the request to the Order Service.



Continued…

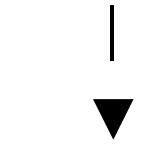
Step 3

The Order Service checks the Inventory Service.



Step 4

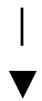
The Order Service contacts the Payment Service.



Step 5

After successful payment,the Order Service sends a message to the Messaging Broker.

Order Service



Messaging Broker

Step 6

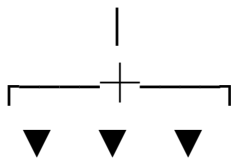
The Messaging Broker sends the message to:

Email Service

SMS Service

Shipping Service

Messaging Broker



Email SMS Shipping

The customer receives an email and SMS while the shipping process begins.

B. Processes Communicating

- **Processes Communicating** refers to the communication between multiple processes in a computer network. Processes can be thought of as individual programs or tasks running on a device, and they may be located on the same device or on different devices connected to the network.
- Communication between processes is facilitated by the use of protocols, which define the rules and formats for exchanging data. The communication between processes can be either synchronous or asynchronous, meaning that either both processes must be available to communicate at the same time or the communication can occur at different times.
- It's important to consider the communication between processes when designing a network application. Factors such as the reliability of communication, the security of communication, and the performance of the communication must be taken into account. The choice of protocols used for communication will depend on the requirements of the application and the network infrastructure.

C. The Interface between the Process and the Computer Network

- **The Interface between the Process and the Computer Network** refers to the connection between a process running on a device and the underlying computer network.
- This interface determines how the process communicates with other processes and with the network itself.
- The interface between a process and the computer network is usually provided by a network stack, which is a collection of protocols and services that handle the communication between the process and the network.
- The network stack translates the process's requests and data into the appropriate network protocols, and vice versa, allowing the process to communicate over the network.

Real-Life Analogy

Imagine sending a **courier package**.

You = Process (Application)

Courier Office = Interface (Socket/API)

Courier Company = Network Stack

Road Network = Internet

Receiver = Another Process

You don't drive the package yourself. Instead, you hand it to the courier office, which takes care of packaging, routing, and delivery.

Example: Opening a Website

Suppose you open:

www.youtube.com

Step 1

- Chrome (process) creates an HTTP request.

Step 2

The request is passed to the **Socket Interface**.

Step 3

The TCP/IP stack adds protocol information:

- TCP Header
- IP Header
- MAC Header

Step 4

- The data is transmitted over the Internet.

Step 5

- The YouTube server receives the request.

Step 6

- The server sends the webpage back through the same interface.

D. Transport Services Available to Applications

- **Transport Services Available to Applications** are the services provided by the network stack that enable applications to communicate with each other over a computer network. These services are responsible for ensuring that data is reliably delivered between applications, and they provide the underlying communication infrastructure for the application.
- There are several transport services available to applications, including:

✓ **TCP (Transmission Control Protocol)**: TCP is a reliable, connection-oriented transport service that provides error-checking and flows control to ensure that data is delivered accurately. Applications that require reliable data delivery, such as email or file transfer, typically use TCP.

✓ **UDP (User Datagram Protocol)**: UDP is an unreliable, connectionless transport service that does not provide error checking or flow control. Applications that require low latency or high speed, such as video streaming or online gaming, typically use UDP.

✓ **SCTP (Stream Control Transmission Protocol)**: SCTP is a reliable, multi-homed transport service that provides error checking and flow control. SCTP can handle multiple streams of data between applications, allowing for efficient communication between applications.

✓ **DCCP (Datagram Congestion Control Protocol)**: DCCP is a transport service that provides congestion control for applications that do not require reliable data delivery.

E. Transport Services Provided by the Internet

The Internet provides two primary transport services for applications: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

1. TCP:

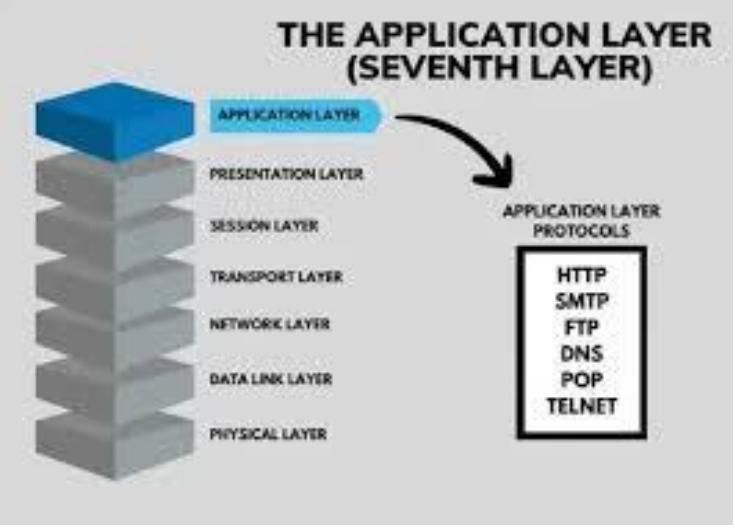
- ✓ TCP is a reliable, connection-oriented transport service that provides error-checking and flows control to ensure that data is delivered accurately.
- ✓ Applications that require reliable data delivery, such as email or file transfer, typically use TCP.
- ✓ TCP establishes a reliable connection between two devices and ensures that data is transmitted in the correct order and without errors.

2. UDP:

- ✓ UDP is an unreliable, connectionless transport service that does not provide error checking or flow control.
- ✓ Applications that require low latency or high speed, such as video streaming or online gaming, typically use UDP.
- ✓ As UDP does not provide error checking or flow control, it is faster and more efficient than TCP, but it may not be suitable for applications that require reliable data delivery.

Application-layer protocols

- **Application-layer protocols** are data exchange protocols used to allow communication between applications on different devices.
- They operate at the highest layer of the OSI (Open Systems Interconnection) model, which is the application layer.
- Application-layer protocols define the rules for exchanging data between applications, such as formatting, error detection and correction, and security.
- Examples of application-layer protocols include HTTP, HTTPS, FTP, SMTP, POP3, IMAP, and many others. They are used in various applications such as web browsing, email, file transfer, and more.



HTTP

- HTTP (Hypertext Transfer Protocol) is a set of rules that govern how information will be transferred between networked devices, specifically web servers and client browsers. HTTP was released in 1991 by Tim Berners-Lee, with the launch of the World Wide Web. Since then, it has evolved and improved for faster and more secure web experiences.

- Evolution of HTTP over the years:-
  - ✓ **HTTP/0.9 (1991):** HTTP was first released in 1991 by Tim Berners-Lee and supported only GET method.
  - ✓ **HTTP/1.0 (1996):** It was released in 1996 and supported many other request methods such as POST, PUT, DELETE, etc. and allowed more complex interaction.
  - ✓ **HTTP/1.1 (1997):** HTTP/1.1 was released in 1997 and offered various new improvements such as pipelining, persistent connections, efficient caching and additional header fields to improve performance and better control.
  - ✓ **HTTP/2 (2015):** HTTP/2 introduced various performance improvements, like, header compression, multiplexing, and server push which helped in reducing the latency.
  - ✓ **HTTP/3 (2020):** HTTP/3 is the latest and currently most used version of HTTP. It aims to improve the performance on unreliable networks. It is based on QUIC protocol and reduces the latency, provides better security and offers faster connection establishment

Evolution of HTTP

1. HTTP/0.9 (1991)

- This was the **first version** of HTTP.

Features

- Supported only the **GET** request method.
- Could retrieve only HTML documents.
- No response headers.
- Very simple protocol.
- Example:

  GET /index.html

- The server simply returned the HTML page.
- **Limitations**
- No POST method
- No images
- No videos
- No status codes
- No error handling
- It was sufficient only for the very early web.

Continued…

2. HTTP/1.0 (1996)

- HTTP/1.0 made the web much more useful.

New Features

It introduced multiple request methods:

- GET
- POST
- PUT
- DELETE
- HEAD

It also added:

- HTTP headers
- Status codes
- MIME types (to transfer images, PDFs, videos)
- Error messages

Continued…

Example response:

HTTP/1.0 200 OK

Content-Type: text/html

Advantages

- Could transfer different file types
- Better interaction between client and server
- Standard response format

Limitation

- Each request required opening a **new TCP connection**, which increased delay.

Continued…

3. HTTP/1.1 (1997)

- HTTP/1.1 became the most widely used version for many years.

Major Improvements

➢ **Persistent Connections (Keep Alive)**

Instead of opening a new TCP connection for every request, one connection could be reused.

Earlier:

Request 1 → New Connection

Request 2 → New Connection

Request 3 → New Connection

HTTP/1.1:

One Connection

↓

Many Requests

This reduced overhead.

➢ Pipelining

The client could send multiple requests without waiting for previous responses.

Example:

- GET image1
- GET image2
- GET image3

➢ Caching

- Frequently accessed webpages could be stored locally.
- Example
- If you revisit a webpage, the browser may load images from its cache instead of downloading them again, reducing bandwidth usage and improving speed.

Continued…

4. HTTP/2 (2015)

- HTTP/2 focused mainly on performance improvements without changing the basic HTTP semantics.

Major Features:

➢ **Multiplexing**

- Multiple requests and responses can travel simultaneously over a **single TCP connection**.
- Instead of waiting for one response before sending the next, many streams are interleaved.
- This significantly reduces page loading time.

➢ **Header Compression**

- HTTP headers often contain repetitive information.
- HTTP/2 uses the **HPACK** compression algorithm to reduce the size of headers, saving bandwidth.

➢ **Server Push**

- The server can proactively send resources before the browser explicitly requests them.

5. HTTP/3 (2020)

- HTTP/3 is the latest version and uses **QUIC** instead of TCP.

What is QUIC?

- QUIC is a transport protocol developed by Google that runs over **UDP** and incorporates features such as reliability, congestion control, and encryption.

Advantages

- **Faster Connection Establishment**

  HTTP/3 combines connection setup and encryption, reducing handshake time.

- **Better Performance on Unstable Networks**

  On Wi-Fi or mobile networks, packet loss has less impact compared to TCP.

- **Reduced Latency**

  Lower delays make web pages load faster.

- **Improved Security**

  Encryption is built in through TLS 1.3.

- **No Head-of-Line Blocking**

  In HTTP/2, if one packet is lost, all streams are delayed because TCP delivers data in order.

  In HTTP/3, streams are independent, so packet loss in one stream does not block others.

Version	Year	Major Features	Limitation Solved
HTTP/0.9	1991	Only GET method	Basic web communication
HTTP/1.0	1996	POST, PUT, DELETE, headers, status codes	Richer client-server communication
HTTP/1.1	1997	Persistent connections, pipelining, caching	Reduced connection overhead
HTTP/2	2015	Multiplexing, header compression, server push	Faster page loading and better bandwidth utilization
HTTP/3	2020	QUIC over UDP, lower latency, improved security, no head-of-line blocking	Better performance on unreliable networks

World Wide Web Communication

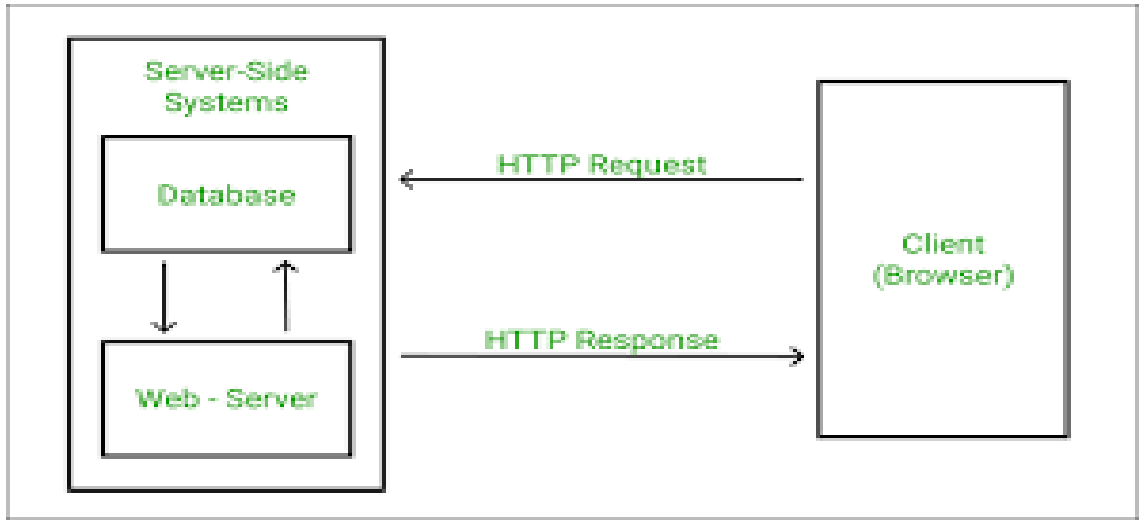
- The World Wide Web is about communication between web **clients** and web **servers**.
- **Clients** are often browsers (Chrome, Edge, Safari), but they can be any type of program or device.
- **Servers** are most often computers in the cloud.



HTTP Request / Response

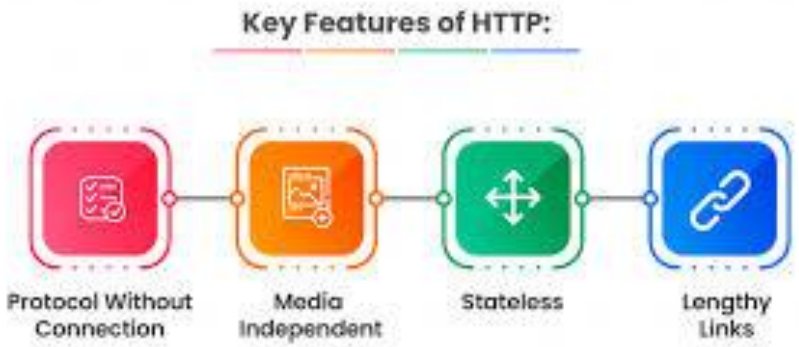
Communication between clients and servers is done by **requests** and **responses**:

1. A client (a browser) sends an **HTTP request** to the web
2. A web server receives the request
3. The server runs an application to process the request
4. The server returns an **HTTP response** (output) to the browser
5. The client (the browser) receives the response



Basic Features of HTTP

- **Stateless:** Each request is treated as a new request i.e both client and server forget about each other after completion of a request unless cookies, tokens or sessions are used.
- **Connectionless:** The connection between client and server is closed after the completion of each request making it a connectionless protocol.
- **Media Independent:** Any type of data can be transferred over web using HTTP protocol, as long as both the client and server specify the format using MIME types.
- **HTTP Methods:** HTTP defines various methods for different actions such as GET, POST, PUT and more.
- **Caching Support:** HTTP provides support for caching which improves the performance by storing the copies of responses and reusing them later



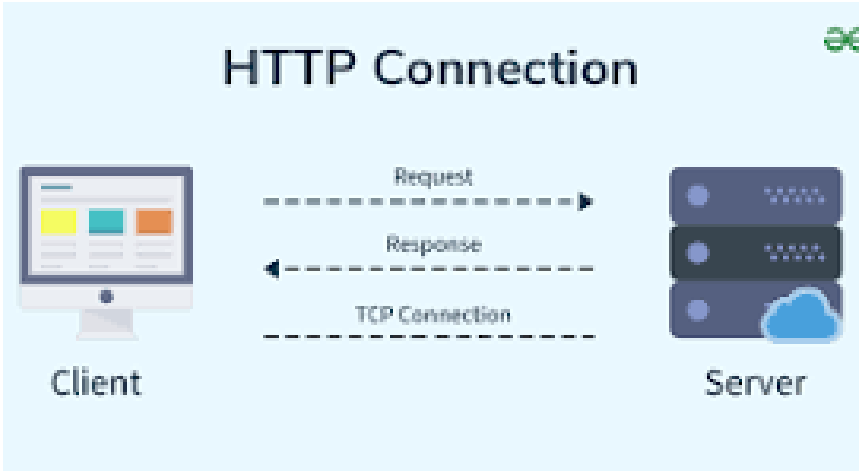
How does HTTP work

- The HTTP protocol transfers resources across the internet between the client devices and servers.
- Client devices ask servers for the resources required to load a web page.
- Then the servers respond by sending the client devices the requested resources.
- Sub-documents, such as information on graphics, text, text layouts, etc., that are shared by requests and responses are put together using a client web browser for showing the entire web page file.

Let us understand ***How does HTTP work*** *with a scenario below:*

Consider the internet like a road. The client, or your home, is at one end of the street. The server, a store where you wish to make a purchase, is at the opposite end of the street. Let us understand the basic technicalities in the scenario now:

- **The web connection:** It allows for online data transmission and reception. It is similar to the roadway between your home and the store.
- **TCP/IP:** These are two simple communication protocols that specify how data should move through the Internet. They are comparable to the transit systems that allow you to place an order, visit the store, and purchase your items. In our scenario, they are comparable to a car or a bike or any commuter that helps you move around.
- **Domain Name System:** A website's address book is analogous to the DNS. When you enter a website address into your browser, it first searches the DNS to determine the IP address of the requested website before retrieving it. For the browser to send HTTP communications to the correct location, it has to determine which server the website is hosted. This is similar to moving around to search for a store's address and find stuff.
- **HTTP:** In the above scenario, it acts as a language for ordering the store's stuff and for communicating between the server and the client.


HTTP Connection
Request
Response
TCP Connection
Client
Server

Step-by-Step Process of Loading a Web Page

- The user sends the “GET” request to the server that hosts the web address once the user fills in the address.
- The GET request, which is made via HTTP, informs the server that the user is trying to find the HTML code that defines the structure and appearance of the login page.
- The login page’s text is contained in the HTML response, while the page’s graphics and videos are requested through separate HTTP requests and the responses.
- It will take longer for the server to answer queries and for the user’s system to load the page when more requests are made, such as when a page with many photos is called.
- TCP/IP is used to compress and transfer information into tiny elements of binary sequences of 1’s and 0’s when these requests and the response pairs are sent. Physical transmission of these packets occurs across electric and fiber-optic cables and wireless networks.

Step-by-Step Process of Loading a Web Page

Step 1: User Enters a URL

Suppose you type the following URL in your browser:

https://www.facebook.com

When you press **Enter**, your browser begins the process of requesting the webpage.

Step 2: Browser Sends an HTTP GET Request

The browser sends an **HTTP GET request** to the web server.

A **GET request** asks the server to send a specific web page or resource.

GET / HTTP/1.1

Host: www.facebook.com

- **GET** → Request to retrieve data.
- **/** → Home page of the website.
- **Host** → Specifies which website is being requested.

Step 3: Server Processes the Request

- The server receives the GET request.
- It locates the requested webpage and prepares a response.
- The response contains the **HTML document** that defines the structure of the webpage.

Example:

```
<html>
<head>
<title>Facebook</title>
</head>
<body>
Login Page
</body>
</html>
```

This HTML tells the browser how to display the page.

Step 4: Browser Receives the HTML Response

The browser downloads the HTML page.

The HTML usually contains references to additional resources such as:

- Images
- CSS files
- JavaScript files
- Videos
- Icons
- Fonts
- Example:

Step 5: Browser Sends Additional HTTP Requests

- For every referenced resource, the browser sends another **HTTP GET request**.

Step 6: Browser Displays the Complete Web Page

After receiving:

- HTML
- Images
- CSS
- JavaScript
- Videos

The browser combines all these resources and displays the complete webpage.

The HTTP protocol is a request/response protocol based on the client/server based architecture where web browsers, robots and search engines, etc. act like HTTP clients, and the Web server acts as a server.

Client

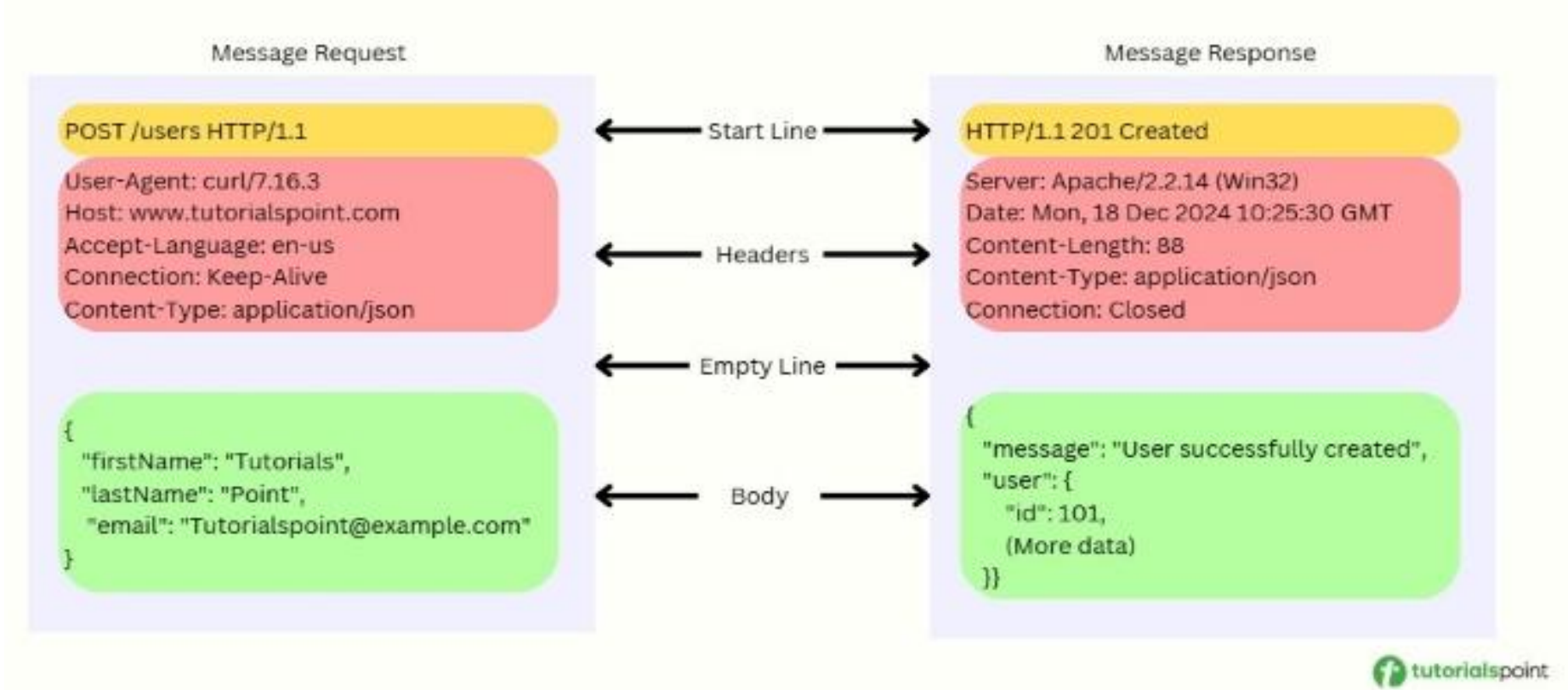
- The HTTP client sends a request to the server in the form of a request method, URI, and protocol version, followed by a MIME-like message containing request modifiers, client information, and possible body content over a TCP/IP connection.

Server

- The HTTP server responds with a status line, including the message's protocol version and a success or error code, followed by a MIME-like message containing server information, entity meta information, and possible entity-body content.

HTTP - Messages

- HTTP Message provides a way for communication between a client and server.
- An HTTP message can be a **request** sent by the client to the server or a **response** sent by the server to the client.
- HTTP is based on the client-server architecture model and a stateless request/response protocol that operates by exchanging messages across a reliable TCP/IP connection.
- An HTTP "client" is a program (Web browser or any other client) that establishes a connection to a server for the purpose of sending one or more HTTP request messages.
- An HTTP "server" is a program (generally a web server like Apache Web Server or Internet Information Services IIS, etc.) that accepts connections in order to serve HTTP requests by sending HTTP response messages


Message Request
Message Response
POST /users HTTP/1.1
Start Line
HTTP/1.1 201 Created
User-Agent: curl/7.16.3
Host: www.tutorialspoint.com
Accept-Language: en-us
Connection: Keep-Alive
Content-Type: application/json
Headers
Server: Apache/2.2.14 (Win32)
Date: Mon, 18 Dec 2024 10:25:30 GMT
Content-Length: 88
Content-Type: application/json
Connection: Closed
Empty Line
{
"firstName": "Tutorials",
"lastName": "Point",
"email": "Tutorialspoint@example.com"
}
Body
{
"message": "User successfully created",
"user": {
"id": 101,
(More data)
}}
tutorialspoint

- HTTP makes use of the Uniform Resource Identifier (URI) to identify a given resource and to establish a connection.
- Once the connection is established, **HTTP messages** are passed in a format similar to that used by the Internet mail and the Multipurpose Internet Mail Extensions (MIME).
- These messages include **requests** from client to server and **responses** from server to client which will have the following format:

HTTP-message = <Request> | <Response> ; HTTP/1.1 messages

HTTP Message Components

HTTP requests and HTTP responses use a generic message format of RFC 822 for transferring the required data. This generic message format consists of the following three items:

1. Message Start Line
2. Headers
3. Message Body

Continued…

1. Message Start Line

A start-line will have the following generic **syntax:**

start-line = Request-Line | Status-Line

- For **request** message, start line contains HTTP method, request-target and HTTP version. Example: GET /hello.htm HTTP/1.1
- For **response** message, start line contains HTTP version, status code and status message. Example: HTTP/1.1 200 OK

Headers

HTTP header fields provide required information about the request or response, or about the object sent in the message body. There are four types of HTTP message headers:

- **General-header:** These header fields have general applicability for both request and response messages.
- **Request-header:** These header fields have applicability only for request messages.
- **Response-header:** These header fields have applicability only for response messages.
- **Entity-header:** These header fields define meta information about the entity-body or, if no body is present, about the resource identified by the request.

Message Body

A message body is the one which carries the actual HTTP request data (including form data and uploaded, etc.) and HTTP response data from the server (including files, images, etc.).

Shown below is the simple content of a message body:

```
<html>
   <body>
        <h1>Hello, World!</h1>
   </body>
 </html>
```

Web Caching

- The activity of saving data for reuse, such as a copy of a web page supplied by a web server, is known as *web caching*.
- It is cached or saved the first time a user accesses the page, and a cache will deliver the copy the next time a user requests the same page, preventing the origin server from becoming overwhelmed.
- Web caching techniques dramatically improve page delivery speed and reduce the amount of work required of the backend server.

Proxy Server

A Proxy Server – an intermediary between the source server and the client – performs web caching. When a client requests information (through an HTTP message), the request is routed through a proxy server, which does the following −

- It first looks to see if the copy is stored locally.
- If it has, it sends the result to the client directly.
- Otherwise, it performs the query on behalf of the end host, saves a copy of the response locally, and sends it back to the end host.

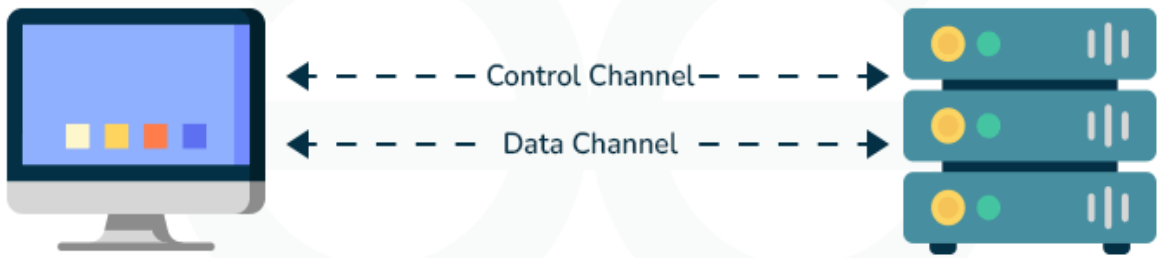
Advantages of Using a Proxy Server

- It reduces the time it takes to answer repeated requests. (Especially if there is a more minor bottleneck between the original server and the recipient than between the proxy server and the receiver.)
- It lowers the cost by reducing the access link bandwidth (at the institution or the office).
- It reduces the overall amount of Internet traffic.

File Transfer Protocol (FTP) in Application Layer

- Located in the application layer of the OSI model, FTP is a basic system that helps in transferring files between a client and a server.
- FTP is a standard communication protocol.
- There are various other protocols like HTTP which are used to transfer files between computers, but they lack clarity and focus as compared to FTP.
- Moreover, the systems involved in connection are heterogeneous, i.e. they differ in operating systems, directories, structures, character sets, etc the FTP shields the user from these differences and transfers data efficiently and reliably.
- FTP can transfer ASCII, EBCDIC, or image files.

FTP



Use Case :- *Upload / Download Files*

There are different ways through which a server and a client do a file transfer using FTP. Some of them are mentioned below:

- ✓**Anonymous FTP**
- ✓**Password Protected FTP**
- ✓**FTP Secure (FTPS)**
- ✓**FTP over Explicit SSL/TLS (FTPES)**
- ✓**Secure FTP (SFTP)**

Anonymous FTP

- Anonymous FTP is enabled on some sites whose files are available for public access.
- A user can access these files without having any username or password.
- Instead, the username is set to anonymous, and the password is to the guest by default.
- Here, user access is very limited. For example, the user can be allowed to copy the files but not to navigate through directories.

Password Protected FTP

- This type of FTP is similar to the previous one, but the change in it is the use of username and password

FTP Secure (FTPS)

- It is also called as FTP Secure Sockets Layer (FTP SSL).
- It is a more secure version of FTP data transfer.
- Whenever FTP connection is established, Transport Layer Security (TLS) is enabled.

FTP over Explicit SSL/TLS (FTPES):

- FTPES helps by upgrading FTP Connection from port 21 to an encrypted connection.

Secure FTP (SFTP):

- SFTP is not a FTP Protocol, but it is a subset of Secure Shell Protocol, as it works on port 22.

What is FTP Useful For?

- **Transferring Large Files**
- **Remote File Management**
- **Automating File Transfers**
- **Accessing Public Files**



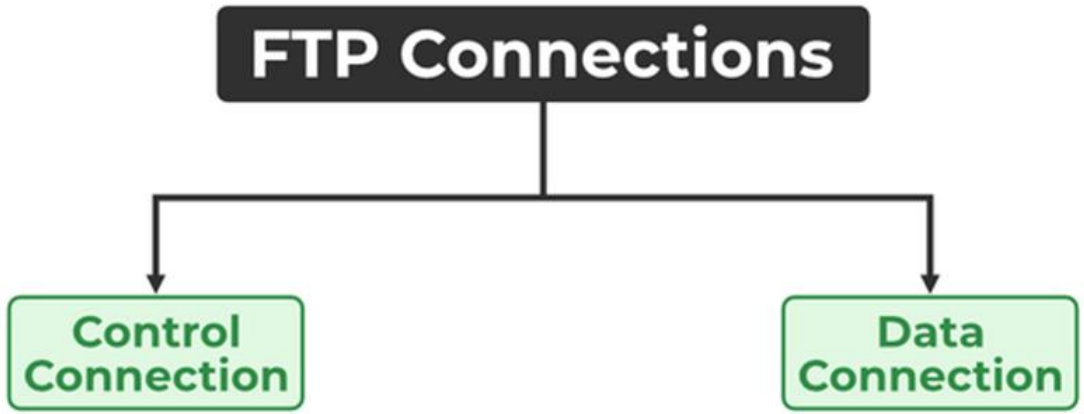
How to Use FTP?

To use FTP, follow these steps:

- **Connect to the FTP Server:** One can connect to the server using the address, username and password through an FTP client or a command line interface. Anonymous Information may not need a username and password.
- **Navigate Directories:** Some commands include ls that is used to list directories and cd that is used to change directories.
- **Transfer Files:** File transfer may be done by using the commands such as get for downloading files, and put for uploading files.
- **Manage Files:** Make operations like deletion (Delete), renaming (Rename) as well as copying (Copy) of files.
- **Close the Connection:** Once file transfer has been accomplished, terminate the connection by giving the bye or quit command.

Types of Connection in FTP

1. Control Connection
2. Data Connection



Control Connection

- For sending control information like user identification, password, commands to change the remote directory, commands to retrieve and store files, etc.,
- FTP makes use of a control connection. The control connection is initiated on port number 21.

Data connection

- For sending the actual file, FTP makes use of a data connection. A data connection is initiated on port number 20.
- FTP sends the control information out-of-band as it uses a separate control

Characteristics of FTP

- FTP uses TCP as a transport layer protocol.
- It is good for simple file transfers, such as during boot time.
- Errors in the transmission (lost packets, checksum errors) must be handled by the TFTP server.
- It uses only one connection through well-known port 69.
- TFTP uses a simple lock-step protocol (each data packet needs to be acknowledged). Thus the throughput is limited.

Advantages of FTP

- **Multiple transfers:** FTP helps to transfer multiple large files in between the systems.
- **Efficiency:** FTP helps to organize files in an efficient manner and transfer them efficiently over the network.
- **Security:** FTP provides access to any user only through user ID and password. Moreover, the server can create multiple levels of access.
- **Continuous transfer:** If the transfer of the file is interrupted by any means, then the user can resume the file transfer whenever the connection is established.
- **Simple:** FTP is very simple to implement and use, thus it is a widely used connection.
- **Speed:** It is the fastest way to transfer files from one computer to another

Disadvantages of FTP

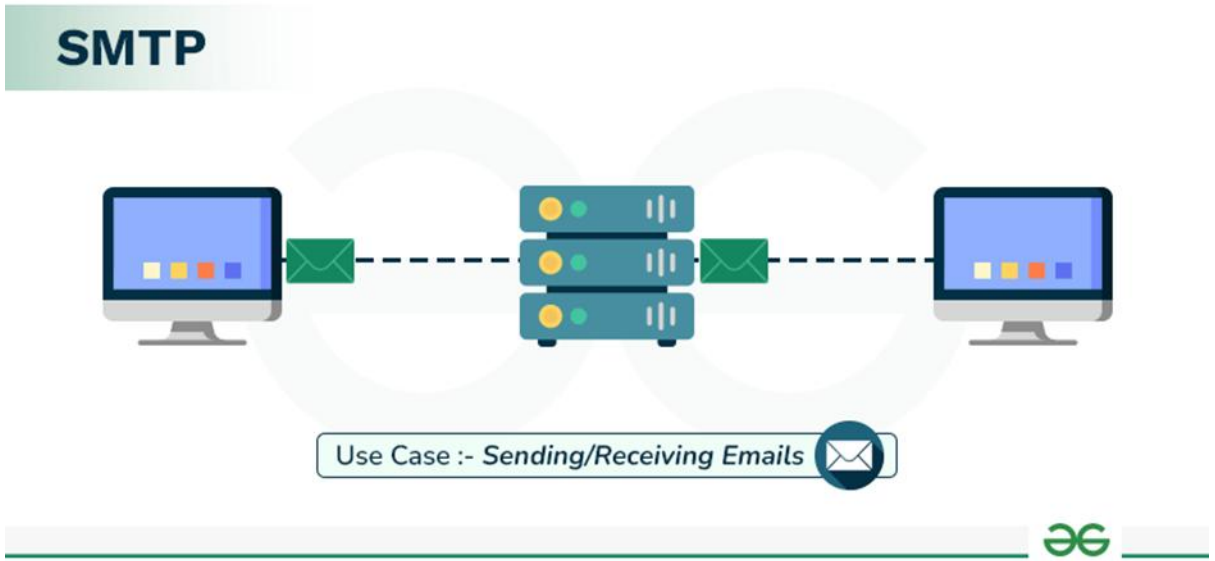
- **Less security:** FTP does not provide an encryption facility when transferring files. Moreover, the username and passwords are in plain text and not a combination of symbols, digits, and alphabets, which makes it easier to be attacked by hackers.
- **Old technology:** FTP is one of the oldest protocols and thus it uses multiple TCP/IP connections to transfer files. These connections are hindered by firewalls.
- **Virus:** The FTP connection is difficult to be scanned for viruses, which again increases the risk of vulnerability.
- **Limited:** The FTP provides very limited user permission and mobile device access**.**
- **Memory and programming:** FTP requires more memory and programming efforts, as it is very difficult to find errors without the commands.

Simple Mail Transfer Protocol (SMTP)

- **Simple Mail Transfer Protocol (SMTP)** is an application layer protocol used for exchanging email messages between servers.
- It is essential in the email communication process and operates at the application layer of the TCP/IP stack.

Continued…

- To send an email, the client opens a TCP connection to the SMTP server.
- The server, which is always listening on port 25, initiates the connection as soon as it detects a client.
- Once the TCP connection is established, the client sends the email across the connection.



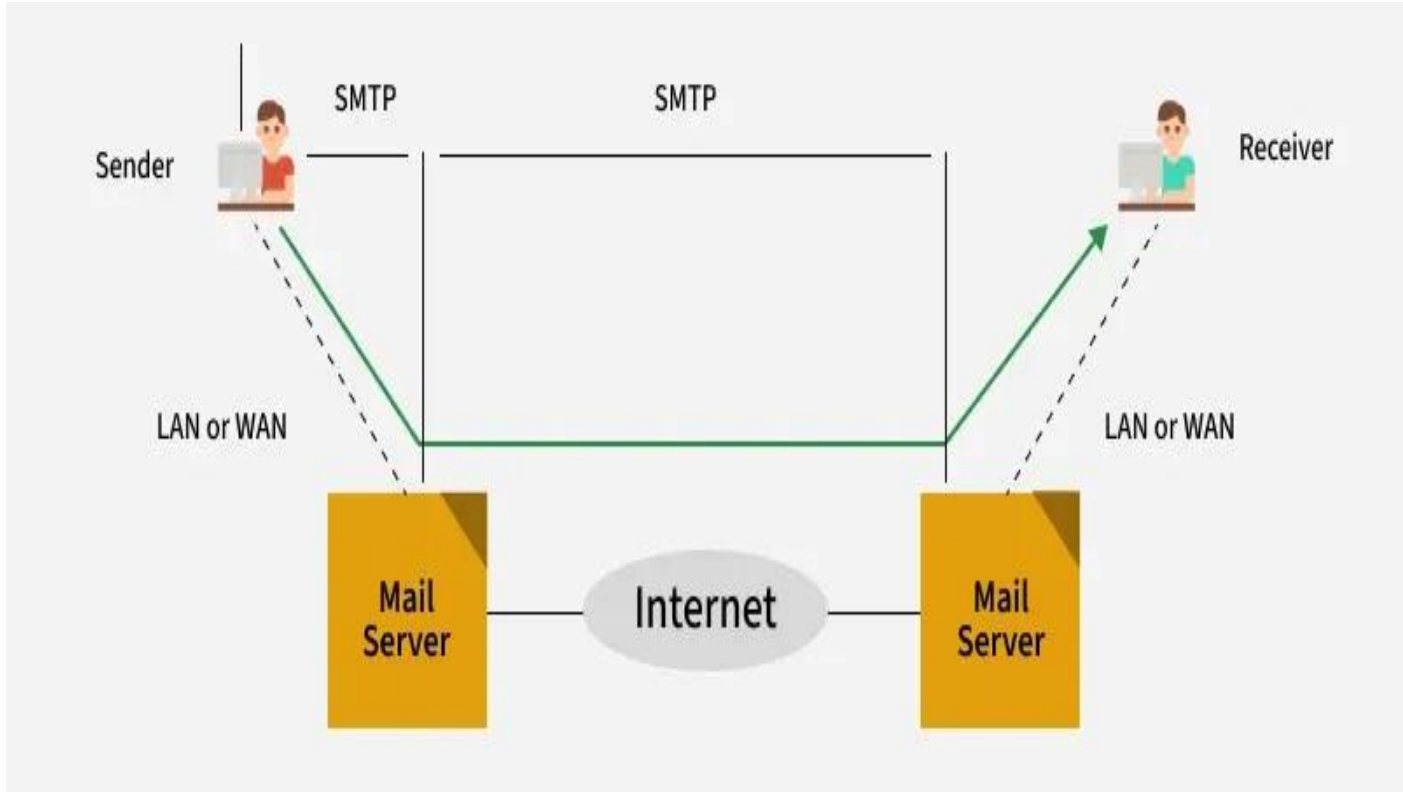
Types of SMTP Protocol

The **SMTP model** supports two types of email delivery methods: **end-to-end** and **store-and-forward**.

- **End-to-end** delivery is used between organizations. In this method, the email is sent directly from the sender's SMTP client to the recipient's SMTP server without passing through intermediate servers.
- **Store-and-forward** is used within organizations that have TCP/IP and SMTP-based networks. In this method, the email may pass through several intermediate servers (Message Transfer Agents, or MTAs) before reaching the recipient.

CONTINUED…

- With **end-to-end** delivery, the SMTP client waits until the email is successfully copied to the recipient's SMTP server before sending it.
- This is different from the **store-and-forward** method, where the email might stop at multiple intermediate servers before reaching its destination.
- In store-and-forward systems, the sender is notified as soon as the email reaches the first server, not the final destination.


SMTP
SMTP
Sender
Receiver
LAN or WAN
LAN or WAN
Mail Server
Internet
Mail Server

Model of SMTP System

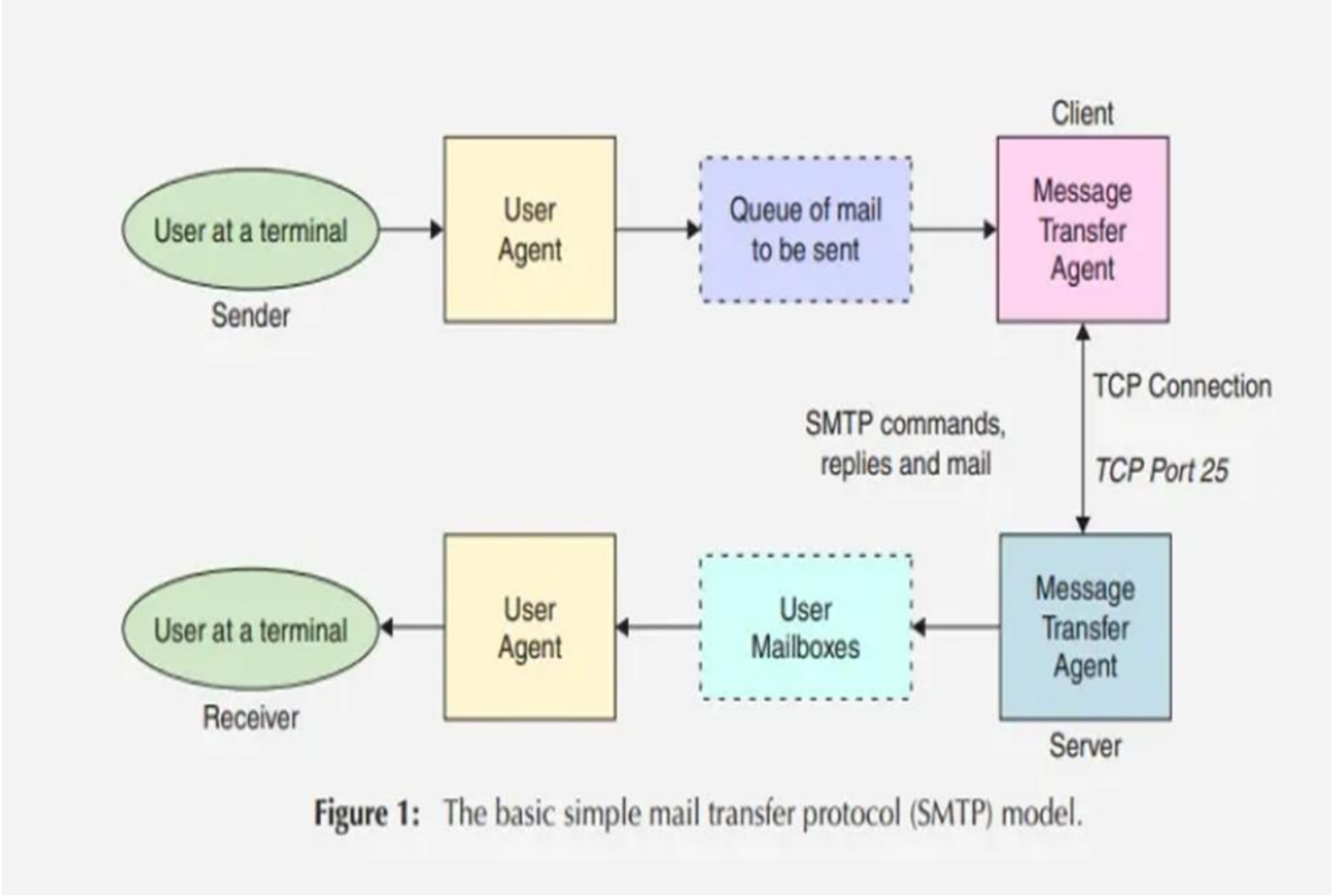


Figure 1: The basic simple mail transfer protocol (SMTP) model.

SMTP Model – Step-by-Step

1. Sender writes an email
The **User at a terminal (Sender)** creates an email using a **User Agent (UA)**.

- A User Agent is an email application such as Gmail, Outlook, Apple Mail, etc. It allows the user to compose, send, read, and manage emails.

2. Email is placed in the mail queue
After the sender clicks **Send**, the message is placed in the **Queue of mail to be sent**.

- The queue temporarily stores outgoing messages until the mail server is ready to transmit them.

3. Client Message Transfer Agent (MTA)
The email is taken from the queue by the **Message Transfer Agent (MTA)**.

- The MTA is responsible for transferring email from one mail server to another.
- Examples of mail-server software include Postfix, Sendmail, and Microsoft Exchange.

4. SMTP communication takes place
The sender's MTA acts as the **SMTP client**, while the receiver's MTA acts as the **SMTP server**.

Contined..

5. Email reaches the receiver's mail server
The receiver-side **Message Transfer Agent** accepts the email and stores it in the appropriate **User Mailbox**.

6. Receiver reads the email
The receiver accesses the mailbox through a **User Agent**.

So the overall flow in your diagram is:

- **Sender → User Agent → Mail Queue → Client MTA → SMTP/TCP → Server MTA → User Mailbox → User Agent → Receiver**

Features of SMTP

1. **Email API**

- Integrating SMTP.com with your business can be easy with the email API feature.
- They have complete API documentation on their website that can help you integrate your business in just 5 minutes.

2. **24x7 Customer Support**:

- The round-the-clock support is one of the best features of SMTP.com.
- Support is available both on the website and also for paid customers. 24x7, all human support is available for all customers across all plans.

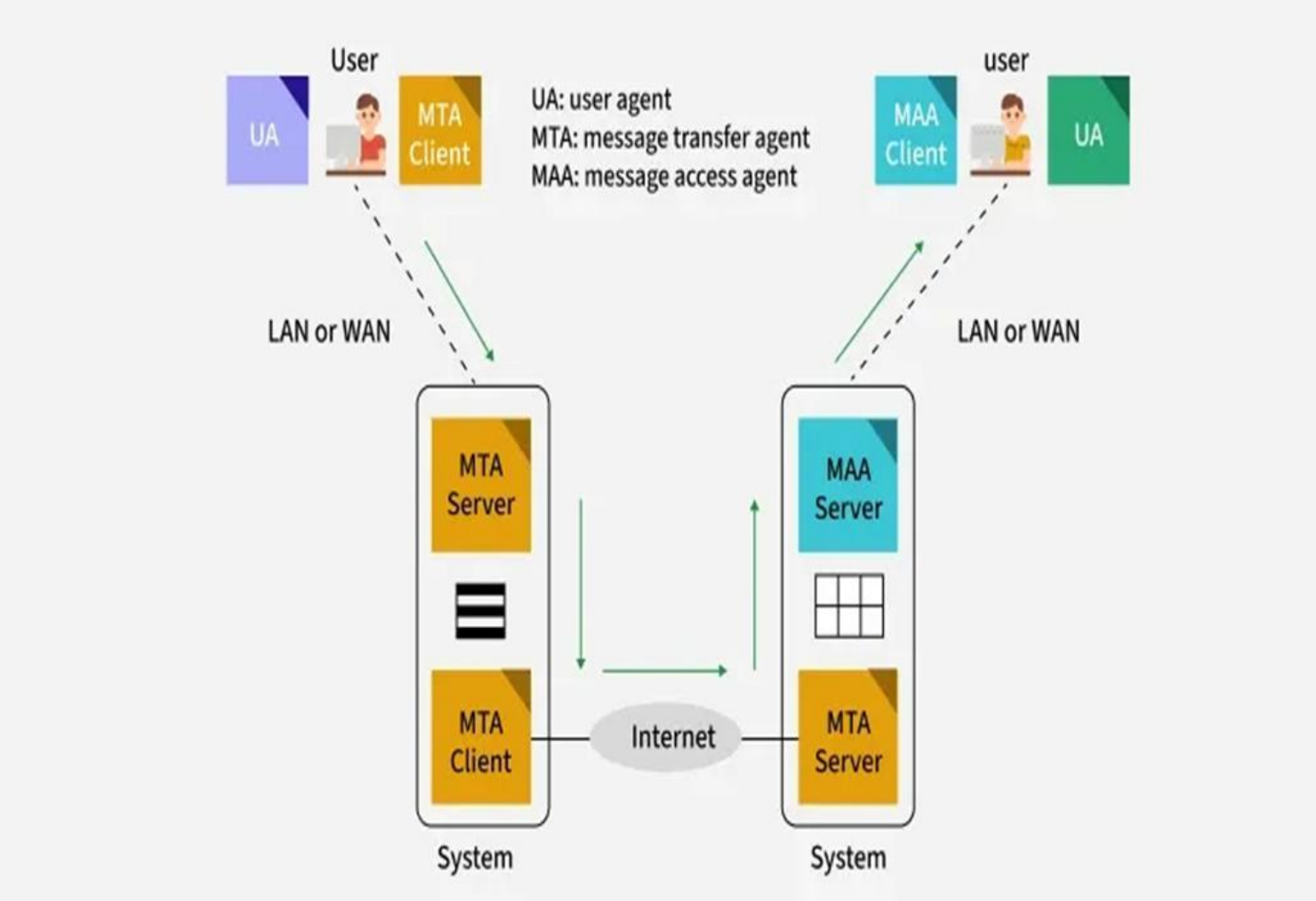
3. High Volume Sending Solutions

- This newly launched feature is great for those businesses who want to send more than 250 million emails a month.
- Customized quotations and solutions are available.

Components of SMTP

- **Mail User Agent (MUA):** It is a computer application that helps you in sending and retrieving mail. It is responsible for creating email messages for transfer to the mail transfer agent(MTA).
- **Mail Submission Agent (MSA):** It is a computer program that receives mail from a Mail User Agent(MUA) and interacts with the Mail Transfer Agent(MTA) for the transfer of the mail.
- **Mail Transfer Agent (MTA):** It is software that has the work to transfer mail from one system to another with the help of SMTP.
- **Mail Delivery Agent (MDA):** A mail Delivery agent or Local Delivery Agent is basically a system that helps in the delivery of mail to the local system.

How does SMTP Work?



1. Sending Email:

- When a user wants to send an email, they use a **User Agent (UA)**, like Outlook or Gmail.
- The email is handed over to the **MTA**, which is responsible for transferring the email to the recipient's mail server.

2. SMTP Client and Server:

- **Sender-SMTP (Client)**: The email sender's MTA initiates the connection to the recipient's MTA (Receiver-SMTP).
- **Receiver-SMTP (Server)**: The receiving MTA listens for incoming connections and receives the email from the sender-SMTP.
- This communication happens over TCP **port 25**.

3. Relays and Gateways:

- **Relays**: In some cases, the email may pass through several intermediate MTAs before reaching the destination server. These MTAs act as **relays**.
- **Gateways**: If the sending and receiving systems use different email protocols (e.g., SMTP and non-SMTP), an **email gateway** can convert the email to the appropriate format for delivery.

4. Email Delivery:

- The sender's **MTA** sends the email to the **receiver's MTA**, either directly or through relays.
- The **MTA** uses the **SMTP** protocol to transfer the message. Once it's delivered to the destination MTA, the email is placed in the recipient's mailbox.
- The recipient's **User Agent (UA)** can then download the email.

SMTP Envelope

Purpose

- The SMTP envelope contains information that guides email delivery between servers.
- It is distinct from the email headers and body and is not visible to the email recipient.

Contents of the SMTP Envelope

- **Sender Address**: Specifies where the email originates.
- **Recipient Addresses**: Indicates where the email should be delivered.
- **Routing Information**: Helps servers determine the path for email delivery

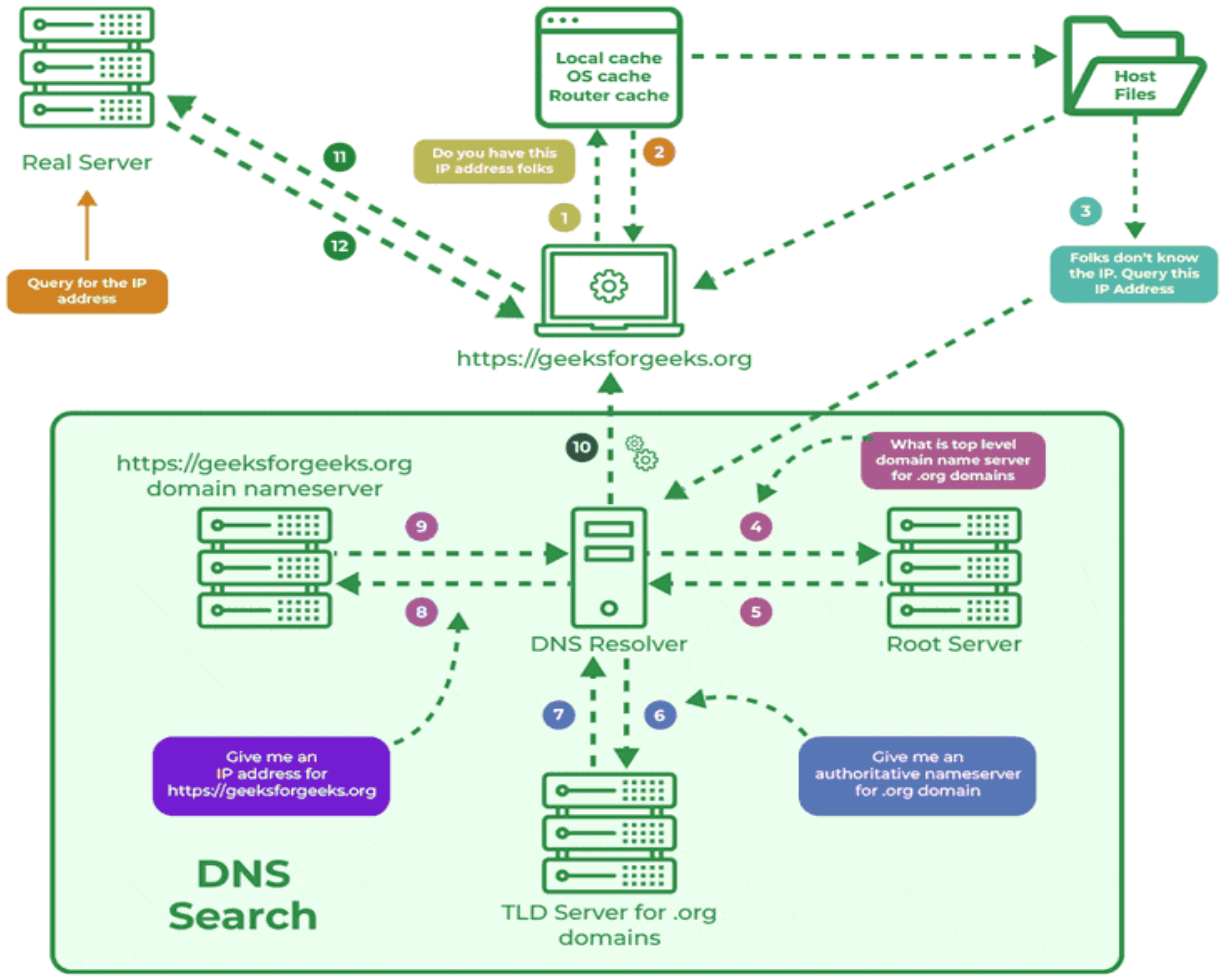
SMTP Ports

- **Port 587**: This is the most commonly used port for **secure SMTP submission** using **TLS (Transport Layer Security)**. It is recommended for client-to-server communication, as it ensures the security of the email transmission.
- **Port 465**: Previously used for **secure SMTP** (SMTPS), this port is **no longer considered an official standard** and is generally not recommended anymore. Many email providers have moved away from port 465 in favour of port 587.
- **Port 25**: This port is traditionally used for **SMTP relay** between mail servers, not for email submission from clients. It is often blocked by ISPs for outgoing mail due to its frequent use for spam and malicious activities.
- **Port 2525**: Although **not an official SMTP port**, it is sometimes used as an alternative for SMTP submission, especially in cases where port 25 is blocked or restricted. Many email providers support this port as an alternative for secure communication.

Domain Name System

- The Domain Name System (DNS) translates human-readable domain names (e.g., www.google.com) into machine-readable IP addresses (e.g., 142.250.190.14), enabling internet communication
- It enables computers to locate and communicate with each other on the internet.
- Functions as a **hierarchical, distributed database**.
- Queries pass through multiple levels:
  - **Root server**
  - **Top-Level Domain (TLD) server**
  - **Authoritative server** (stores the specific IP address).
- Ensures seamless website access using easy-to-remember names instead of numerical IP addresses.

How Does DNS Works

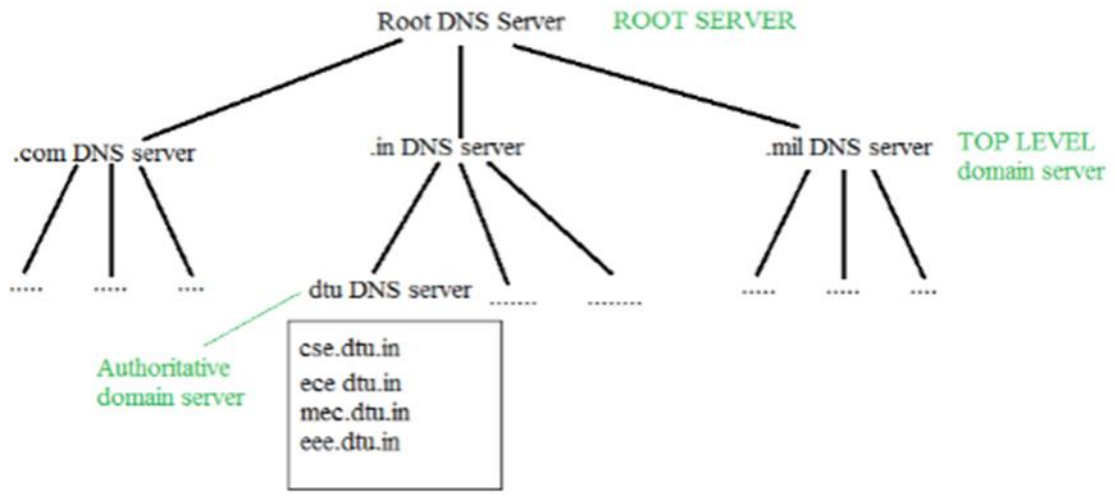


How Does DNS Work?

- When we type a website like https://www.sitpune.edu.in in our browser, our computer tries to find the IP address.
- First, it checks the local cache (our browser, operating system, or router) to see if it already knows the IP address.
- If the local cache doesn’t have the IP, the query is sent to a DNS resolver to find it.
- DNS resolver may check host files (used for specific manual mappings), but usually, it moves on.
- Resolver sends the query to a Root DNS server, which doesn't know the exact IP address but points to the TLD server (e.g., .org server for this example).
- TLD server then directs the resolver to the authoritative nameserver for www.sitpune.edu.in.
- Authoritative nameserver knows the exact IP address for www.sitpune.edu.in and sends it back to the resolver.
- Resolver passes the IP address to our computer.
- Our computer uses the IP address to connect to the real server where the website is hosted.
- The website loads in our browser.

Structure of DNS

It is very difficult to find out the IP address associated with a website because there are millions of websites and with all those websites we should be able to generate the IP address immediately, there should not be a lot of delays for that to happen organization of the database is very important.



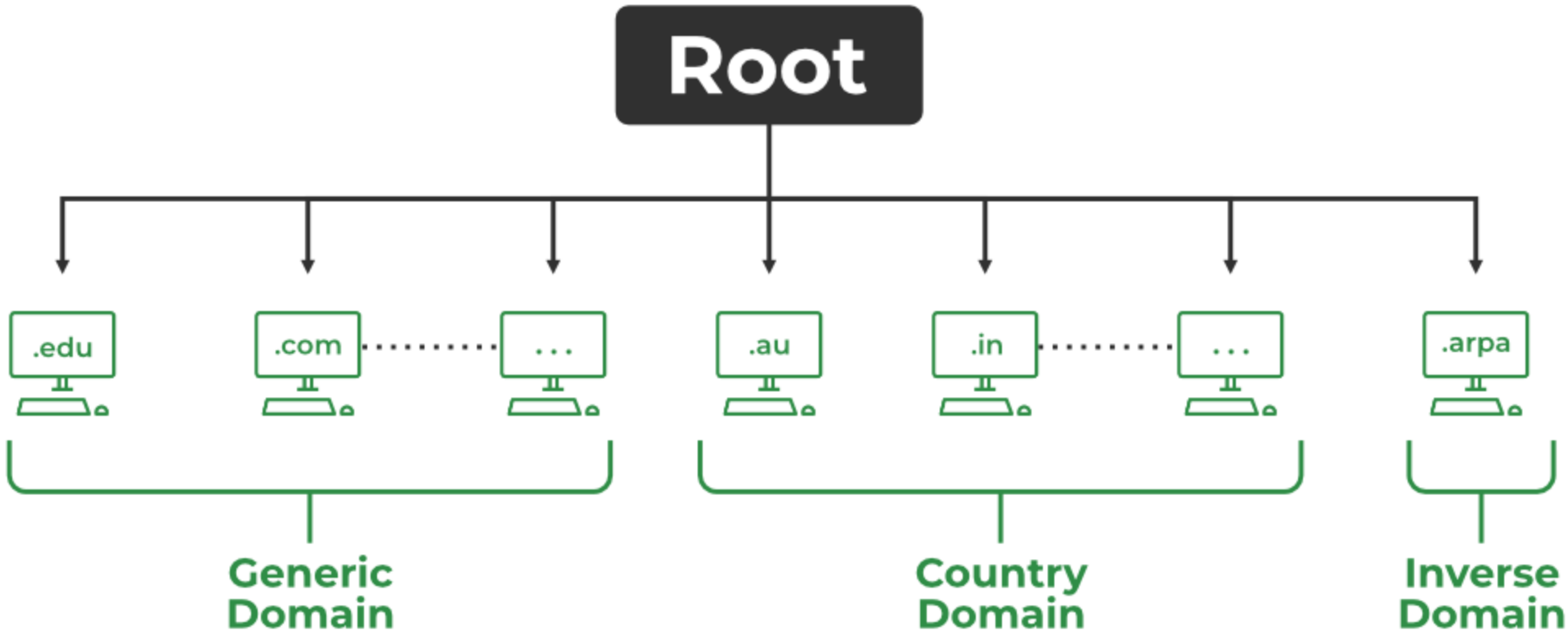
There are various kinds of domains:

- **Generic Domains:** .com(commercial), .edu(educational), .mil(military), .org(nonprofit organization), .net(similar to commercial) all these are generic domains.

- **Country Domain:** .in (India) .us .uk

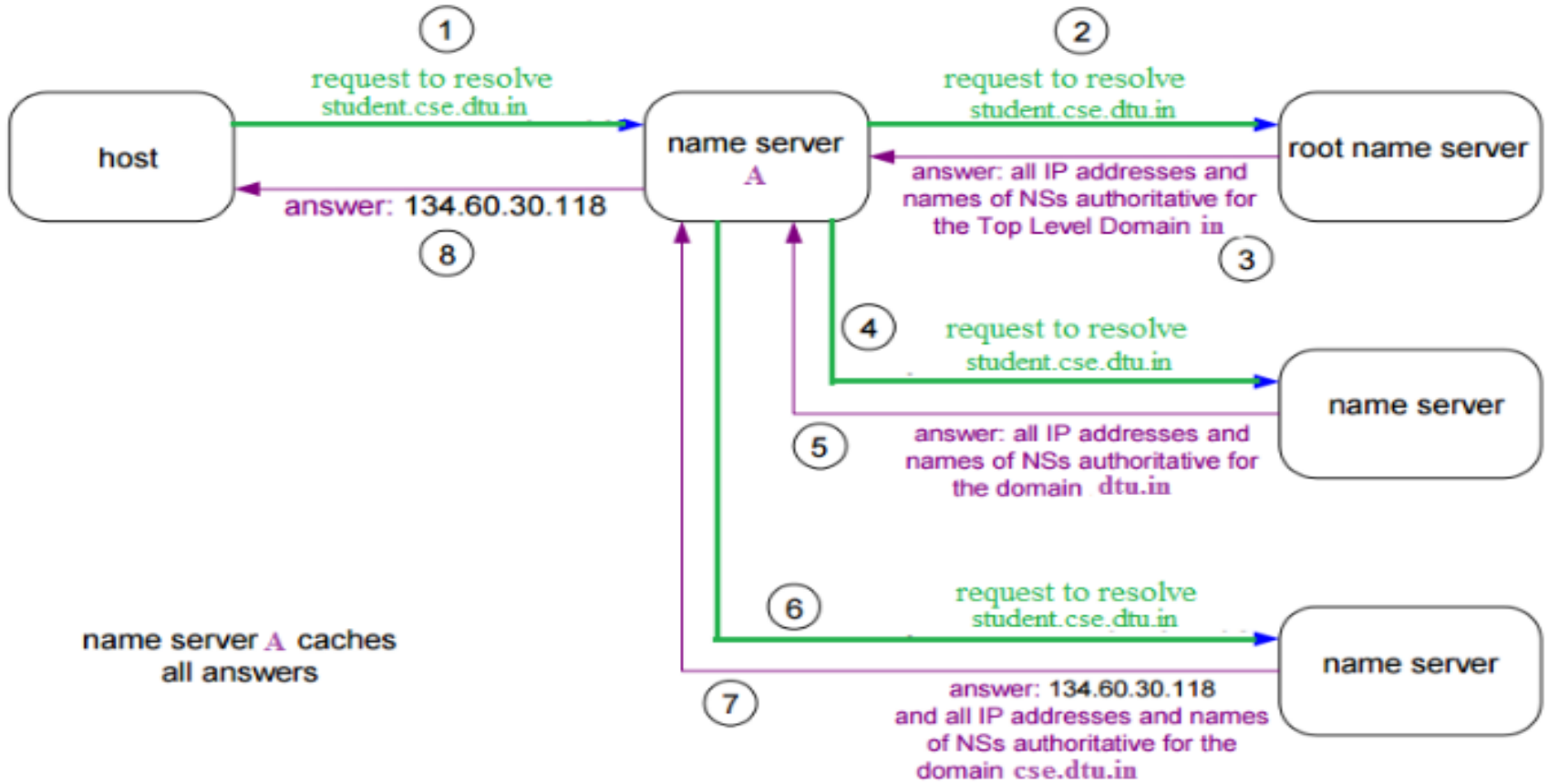
- **Inverse Domain:** if we want to know what is the domain name of the website. IP to domain name mapping. So DNS can provide both the mapping for example to find the IP addresses of *sitpune.edu.in* then we have to type:

nslookup www.sitpune.edu.in


Root
.edu
.com
. . .
.au
.in
. . .
.arpa
Generic Domain
Country Domain
Inverse Domain

Domain Name Server

- The client machine sends a request to the local name server, which, if the root does not find the address in its database, sends a request to the root name server, which in turn, will route the query to a top-level domain (TLD) or authoritative name server.
- The root name server can also contain some hostName to IP address mappings.
- The Top-level domain (TLD) server always knows who the authoritative name server is.
- So finally the IP address is returned to the local name server which in turn returns the IP address to the host.


1
request to resolve
student.cse.dtu.in
host
name server
A
2
request to resolve
student.cse.dtu.in
root name server
answer: all IP addresses and
names of NSs authoritative for
the Top Level Domain in
3
answer: 134.60.30.118
8
4
request to resolve
student.cse.dtu.in
name server
5
answer: all IP addresses and
names of NSs authoritative for
the domain dtu.in
6
request to resolve
student.cse.dtu.in
name server
name server A caches
all answers
7
answer: 134.60.30.118
and all IP addresses and names
of NSs authoritative for the
domain cse.dtu.in

DNS Lookup

DNS Look up also called DNS Resolution, is the process of translating a human-readable domain name (like www.example.com) into its corresponding IP address (like 192.0.2.1), which computers use to locate and communicate with each other on the internet. It allows users to access websites easily using names instead of remembering numeric IP addresses.

- DNS Lookup starts when a user types a domain name into their browser.
- The query goes through a series of servers: the DNS resolver, Root server, TLD server and authoritative server.
- Each server plays a role in finding the correct IP address for the domain.
- Once the IP address is found, the browser connects to the website’s server and loads the page.

DNS Resolver

- DNS Resolver is simply called a DNS Client and has the functionality for initiating the process of DNS Lookup which is also called DNS Resolution.
- By using the DNS Resolver, applications can easily access different websites and services present on the Internet by using domain names that are very much friendly to the user and that also resolves the problem of remembering IP Address.

Types of DNS Queries

There are basically three types of DNS Queries that occur in DNS Lookup. These are stated below.

- **Recursive Query:** In this query, if the resolver is unable to find the record, in that case, DNS client wants the DNS Server will respond to the client in any way like with the requested source record or an error message.
- **Iterative Query:** Iterative Query is the query in which DNS Client wants the best answer possible from the DNS Server.
- **Non-Recursive Query:** Non-Recursive Query is the query that occurs when a DNS Resolver queries a DNS Server for some record that has access to it because of the record that exists in its cache.

DNS Caching and TTL (Time-to-Live)

- **DNS Caching** is a mechanism that stores DNS records locally to avoid querying external DNS servers repeatedly for the same information. This speeds up the browsing experience and reduces network traffic.
- TTL (Time-to-Live) is the amount of time that a DNS record is cached before it expires. When the TTL expires, the cache is cleared and a fresh DNS query must be made. The TTL is defined by the authoritative DNS server, which can adjust this based on the nature of the domain.
- **For example**, if the TTL for www.sitpune.edu.in is set to 3600 seconds (1 hour), then the DNS record will be stored in the cache for one hour before it expires and requires a new lookup.

P2P (Peer To Peer) File Sharing

- P2P (Peer-to-Peer) is a file-sharing technology, that allows users to access mainly the multimedia files like videos, music, e-books, games, etc.

- The individual users in this network are referred to as peers. The peers request files from other peers by establishing TCP or UDP connections.

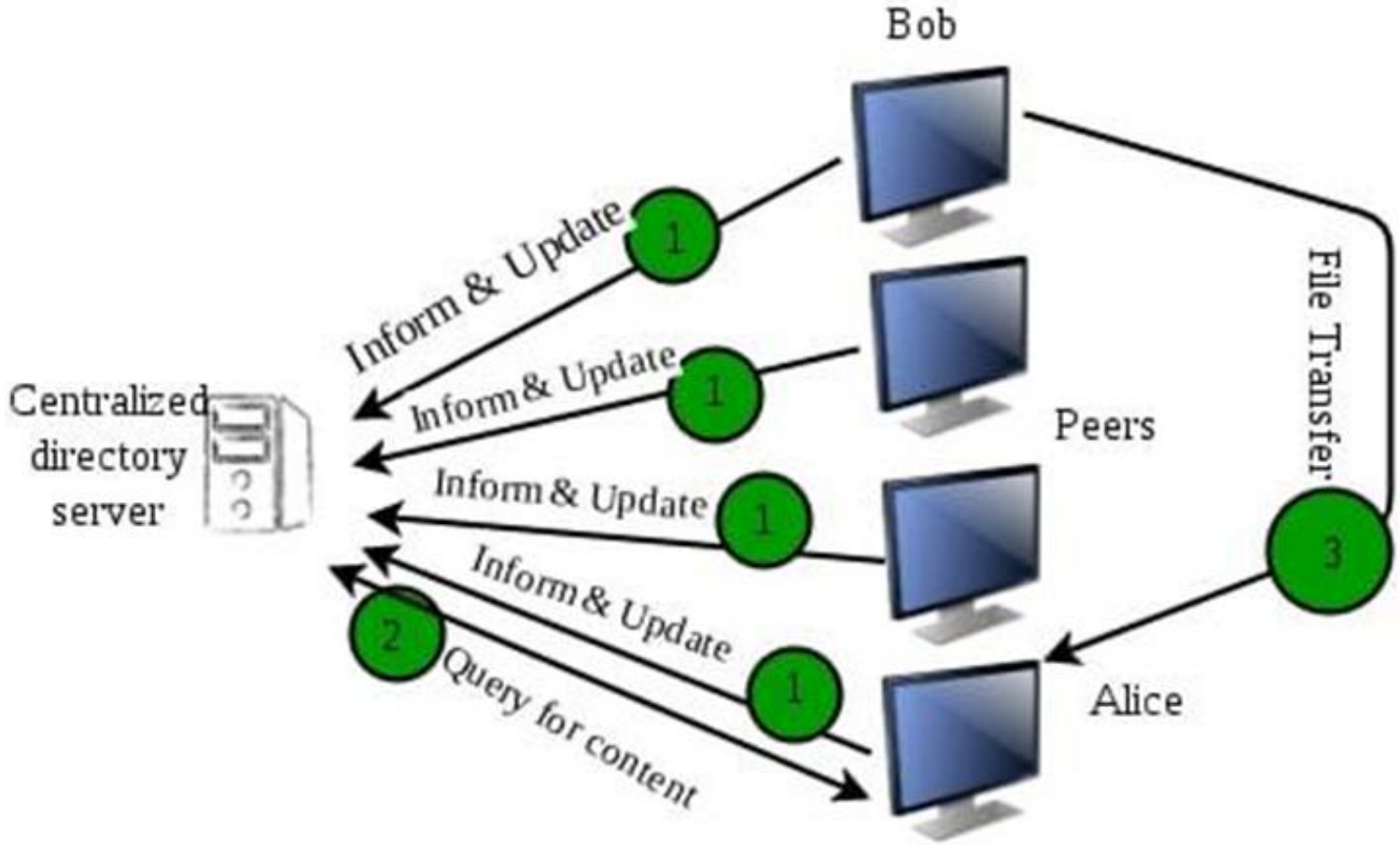
How Does P2P (Peer-to-Peer) Work?

- A peer-to-peer network allows computer hardware and software to communicate without the need for a server.
- Unlike client-server architecture, there is no central server for processing requests in a P2P architecture. The peers directly interact with one another without the requirement of a central server.
- Now, when one peer makes a request, multiple peers may have a copy of that requested object.
- Now the problem is how to get the IP addresses of all those peers. This is decided by the architecture supported by the P2P systems.

Using one of these methods, the client peer can get to know all the peers which have the requested object/file and the file transfer takes place directly between these two peers.

Various methods in P2P Architecture

- **Centralized Directory**
  - A centralized Directory is somewhat similar to client-server architecture in the sense that it maintains a huge central server to provide directory service.
  - All the peers inform this central server of their IP address and the files they are making available for sharing.
  - The server queries the peers at regular intervals to make sure if the peers are still connected or not.
  - So basically this server maintains a huge database regarding which file is present at which IP addresses. The first system which made use of this method was **Napster**, for Mp3 distribution.
  - Now whenever a requesting peer comes in, it sends its query to the server.
  - Since the server has all the information of its peers, so it returns the IP addresses of all the peers having the requested file to the peer.
  - Now the file transfer takes place between these two peers.



P2P paradigm with a centralised directory

Problem in Centralized Directory

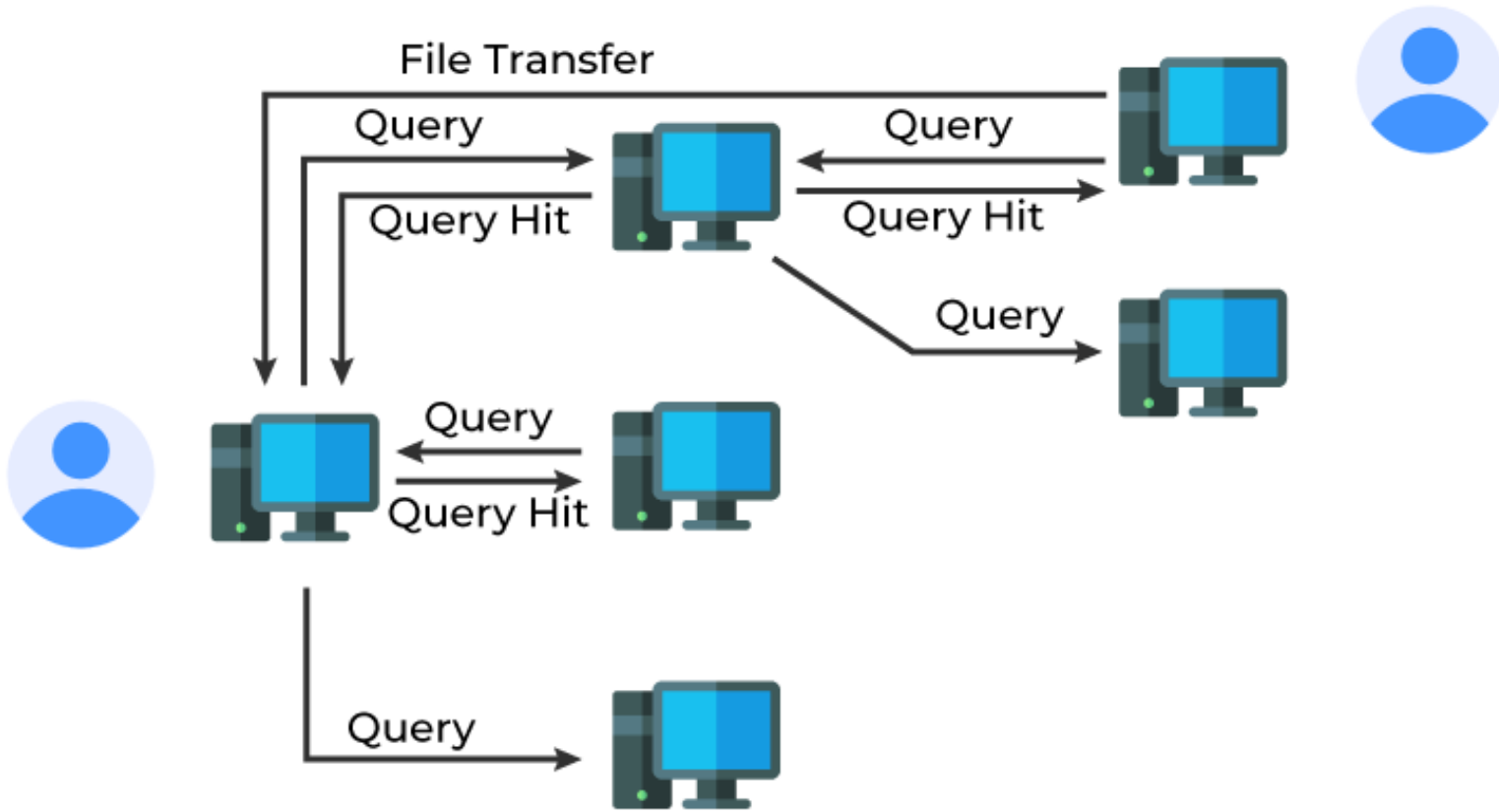
- The major problem with such an architecture is that there is a single point of failure.
- If the server crashes, the whole P2P network crashes.
- Also, since all of the processing is to be done by a single server so a huge amount of the database has to be maintained and regularly updated.

2. Query Flooding

- Unlike the centralized approach, this method makes use of distributed systems.
- In this, the peers are supposed to be connected to an overlay network.
- It means if a connection/path exists from one peer to another, it is a part of this overlay network.
- In this overlay network, peers are called nodes, and the connection between peers is called an edge between the nodes, thus resulting in a graph-like structure.
- **Gnutella** was the first decentralized peer-to-peer network.

Working

- Now when one peer requests for some file, this request is sent to all its neighboring nodes i.e. to all nodes connected to this node.
- If those nodes don't have the required file, they pass on the query to their neighbors and so on. This is called query flooding.
- When the peer with the requested file is found (referred to as query hit), the query flooding stops and it sends back the file name and file size to the client, thus following the reverse path.
- If there are multiple query hits, the client selects from one of these peers.


File Transfer
Query
Query
Query Hit
Query Hit
Query
Query
Query Hit
Query

Advantages

- **Simple and deterministic**: If the resource exists within reach, flooding ensures discovery across reachable nodes .

- **No central coordination**: Fully decentralized, with no single point of failure

Drawbacks

- **Highly inefficient and unscalable**: Each query propagates exponentially, overwhelming the network with traffic. As network size grows, cost balloons dramatically .

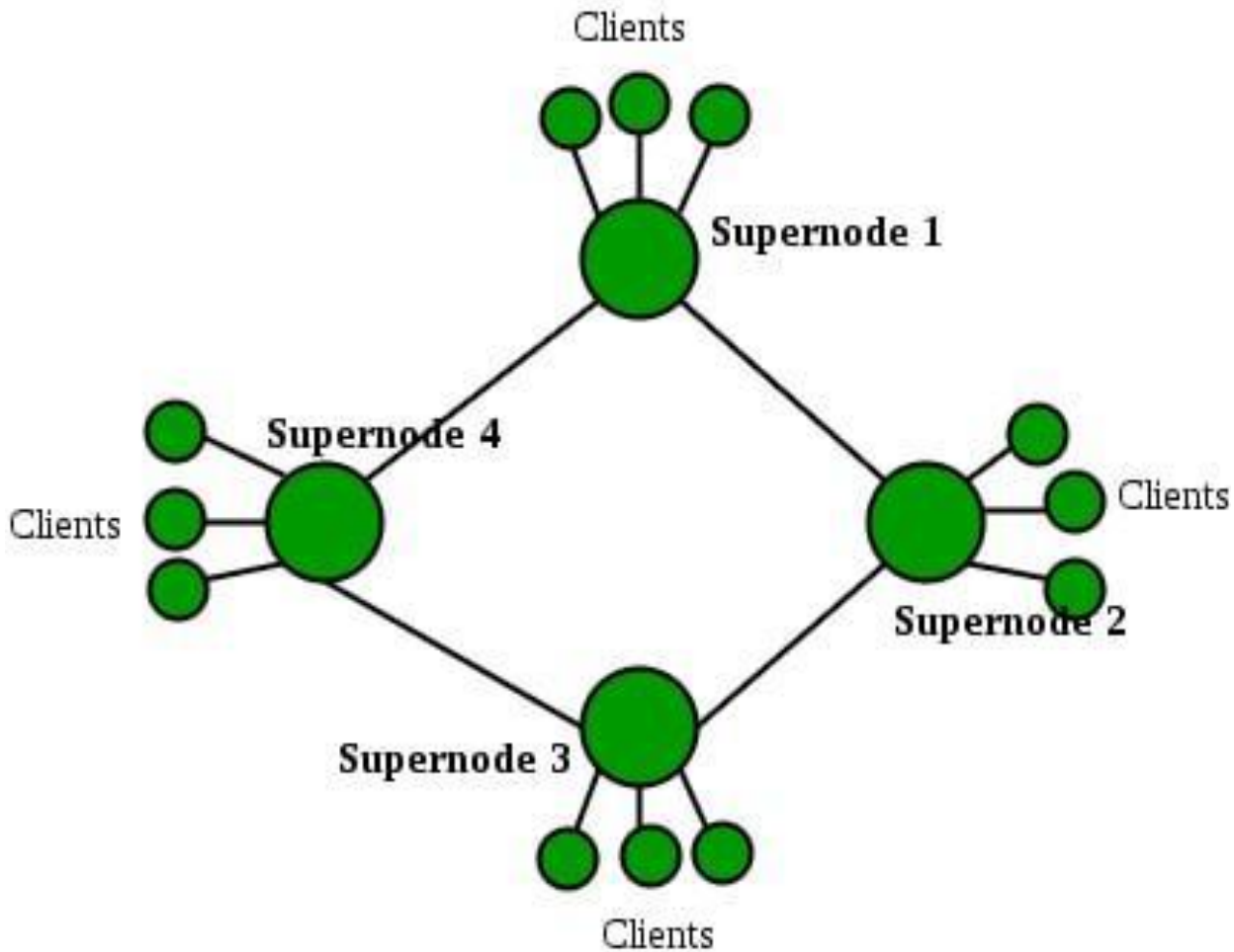
- **High resource consumption**: Every peer must process and forward every query, leading to bandwidth, CPU, and memory overload

3. Exploiting Heterogeneity

- This P2P architecture makes use of both the above-discussed systems.
- It resembles a distributed system like Gnutella because there is no central server for query processing.
- But unlike Gnutella, it does not treat all its peers equally.
- The peers with higher bandwidth and network connectivity are at a higher priority and are called **group leaders/supernodes**.
- The rest of the peers are assigned to these supernodes.
- These supernodes are interconnected and the peers under these supernodes inform their respective leaders about their connectivity, IP address, and the files available for sharing.
- **KaZaA** technology is such an example that makes use of Napster and Gnutella

Working

- This structure can process the queries in two ways.
- The first one is that the supernodes could contact other supernodes and merge their databases with their database. Thus, this supernode now has information about a large number of peers.
- Another approach is that when a query comes in, it is forwarded to the neighboring super nodes until a match is found, just like in Gnutella.
- Thus query flooding exists but with limited scope as each supernode has many child peers.
- Hence, such a system exploits the heterogeneity of the peers by designating some of them as group leaders/supernodes and others as their child peers


Clients
Supernode 1
Supernode 4
Clients
Clients
Supernode 2
Supernode 3
Clients

Socket Programming in Computer Network

- **Sockets** in computer networks are used for allowing the transmission of information between two processes of the same machines or different machines in the network.
- The socket is the combination of IP address and software port number used for communication between multiple processes.
- Socket helps to recognize the address of the application to which data is to be sent using the IP address and port number.

Continued…

- Sockets are mostly used in client-server architecture for communication between multiple applications.
- Socket programming tells us how we can use socket API for creating communication between local and remote processes.
- The socket is created by the combination of the IP address and port number of the software. With this combination, the process knows the system address and address of the application where data is to be sent.
- **:** is used to separate IP address and port number. For eg: 192.168.1.67:80, 155.2.12.23:77, etc.

Socket Types:

- **TCP Socket (Stream Socket):** Provides reliable, connection-based communication (i.e., TCP Protocol).
- **UDP Socket (Datagram Socket):** Provides connectionless communication, faster but unreliable (i.e., UDP Protocol).

Client-Server Model

- The client- server model refers to the architecture used in socket programming, where a client and a server to interact with each other to exchange information or services.
- This architecture allows client to send service requests and the server to process and send response to those service requests.